

Přehled základních kompresních technik

Zdeněk VAŠÍČEK

Fakulta informačních technologií, Vysoké učení technické v Brně

Brno, Czech Republic

vasicek@fit.vutbr.cz

Obsah

- **Metody intuitivní komprese dat**
 - Komprese textu s omezeným množstvím symbolů
 - Komprese slovníkových dat

- **RLE (Run-Length encoding)**
 - princip RLE kódování a jeho vlastnosti
 - použití RLE pro kompresi textu, binárních dat a obrazů

- **Transformace**
 - MTF (Move-to-Front Transformation)
 - BWT (Burrows-Wheeler Transformation)

Stručná historie komprese dat

- Komprese dat může být chápána jako umění tvorby kompaktnějších reprezentací, které má kořeny hluboko v historii.
 - 1000BC, těsnopis (shorthand)
 - 1829, Brailův kód
 - 1838, Morseův kód
 - 1888, Baudotův kód (telegraf)
 - 1951, Huffmanovo kódování
 - 1967, Kódování délek sledů (Run-length encoding), Adaptivní Huffmanovo kódování
 - 1975, Aritmetické kódování
 - 1977, Metody slovníkové komprese
 - 1984, Metoda slovníkové komprese LZW (zip, HW implementace)
 - 80-tá léta – BMP, TIFF (FAX)
 - 90-tá léta – GIF, JPEG standard (JBIG), motion video compression (MPEG), satelitní TV
 - 20-té století – digitální TV, DVD, MP3, HDTV
 - 2014, kódování Finited State Entropy

Bezeztrátové algoritmy pro kompresi textu

- Ukázka kompresní výkonnosti bezeztrátových algoritmů pro kompresi textu (Calgary Corpus)

Rok	bpc	Algoritmus	Autoři
1977	3.94	LZ77	Ziv, Lempel
1984	3.32	LZMW	Miller, Wegman
1987	3.30	LZH	Brent
1987	3.24	MTF (MtF , Huffman?)	Moffat
1987	3.18	LZB	Bell
1988	2.48	PPMC	Moffat
1992	2.71	GZIP (LZ77 , Shannon-Fano, Huffman)	Gailly, Adler
1994	2.34	PPM*	Cleary, Teahan, Witten
1995	2.33	PPMD	Moffat
1995	2.04	ACB v1.14 (LZ variant)	Buyanovsky
1996	2.19	BZIP2 (Burrows-Wheeler , Huffman)	Burrows, Wheeler
1997	1.99	BOA (PPM variant)	Sutton
1999	1.89	RKIVE (Reduced-offset LZ, PPMZ)	Taylor

BOA/RKIVE – varianty PPM, které predikují znaky podle statistiky n-gramů

web.archive.org/web/20000622221015/http://www.cs.waikato.ac.nz/~singlis/ratios.html



Kódování znaků

Kód Kamenický

##	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
8.	Č 10C	ü FC	é E9	d' 10F	ä E4	Ď 10E	Ť 164	č 10D	ě 11B	Ě 11A	Ĺ 139	Í CD	Ĳ 13E	Í 13A	Ä C4	Á C1
9.	É C9	ž 17E	Ž 17D	ô F4	ö F6	Ó D3	û 16F	Ú DA	ý FD	Ö D6	Ü DC	Š 160	Ľ 13D	Ý DD	Ř 158	ť 165
A.	á E1	í ED	ó F3	ú FA	ň 148	Ň 147	Û 16E	Ô D4	š 161	ř 159	í 155	Ř 154	¼ BC	§ A7	« AB	» BB
B.	☼ 2591	☼ 2592	☼ 2593	 2502	 2524	 2561	 2562	 2556	 2555	 2563	 2551	 2557	 255D	 255C	 255B	 2510
C.	Ł 2514	Ł 2534	Ł 252C	Ł 251C	Ł 2500	Ł 253C	Ł 255E	Ł 255F	Ł 255A	Ł 2554	Ł 2569	Ł 2566	Ł 2560	Ł 2550	Ł 256C	Ł 2567
D.	Ł 2568	Ł 2564	Ł 2565	Ł 2559	Ł 2558	Ł 2552	Ł 2553	Ł 256B	Ł 256A	Ł 2518	Ł 250C	Ł 2588	Ł 2584	Ł 258C	Ł 2590	Ł 2580
E.	α 3B1	β DF	Γ 393	π 3C0	Σ 3A3	σ 3C3	μ B5	τ 3C4	Φ 3A6	Θ 398	Ω 3A9	δ 3B4	∞ 221E	φ 3C6	ε 3B5	∩ 2229
F.	≡ 2261	± B1	≥ 2265	≤ 2264	∫ 2320	∫ 2321	÷ F7	≈ 2248	° B0	· 2219	· B7	√ 221A	² 207F	² B2	■ 25A0	A0

KOI8-CS (ČSN)

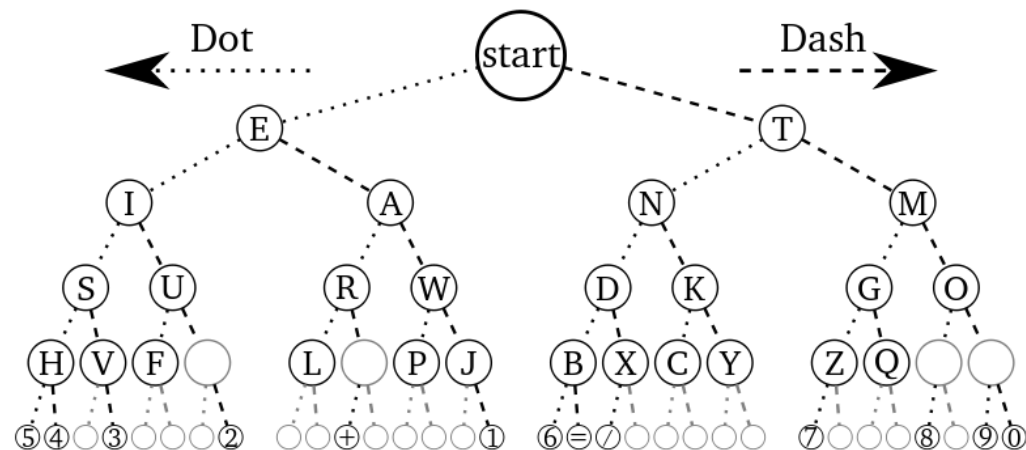
##	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
C.	à 00E0	á 00E1	ă 01CE	č 010D	d' 010F	ě 011B	í 0155	ch 0063 0068	ü 00FC	í 00ED	û 016F	Í 013A	Ĳ 013E	ö 00F6	ň 0148	ó 00F3
D.	ô 00F4	ä 00E4	ř 0159	š 0161	ť 0165	ú 00FA	ě 00EB	é 00E9	û 0171	ý 00FD	ž 017E			ö 0151	è 0117	ß 00DF
E.	À 00C0	Á 00C1	Ä 01CD	Č 010C	Ď 010E	Ě 011A	Ř 0154	Ch 0043 0068	Ü 00DC	Í 00CD	Û 016E	Ĺ 0139	Ľ 013D	Ö 00D6	Ň 0147	Ó 00D3
F.	Ô 00D4	Ä 00C4	Ř 0158	Š 0160	Ť 0164	Ú 00DA	Ě 00CB	É 00C9	Û 0170	Ý 00DD	Ž 017D			Ö 0150	È 0116	

Windows-1250

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
8x	€	NZ	,	NZ	„	...	†	‡	NZ	‰	Š	Š	Š	Š	Ž	Ž
9x	NZ	‘	’	“	”	•	—	—	NZ	™	š	š	š	š	ž	ž
Ax	NBSP	˘	˘	Ł	Ł	Ł	Ł	Ł	Ł	©	Š	«	»	SHY	®	Ž
Bx	°	±	•	Ł	Ł	Ł	Ł	Ł	Ł	Ł	š	»	Ł	Ł	Ł	ž
Cx	Ř	Á	Â	Ă	Ä	Í	Ć	Ç	Č	É	Ę	Ě	Ě	í	î	Ď
Dx	Đ	Ń	Ń	Ó	Ô	Õ	Ö	×	Ř	Ú	Ú	Ů	Ü	Ý	Ť	ß
Ex	í	á	â	ă	ä	í	ć	ç	č	é	ę	ě	ě	í	î	ď
Fx	đ	ń	ń	ó	ô	õ	ö	÷	ř	ú	ú	ů	ü	ý	ț	•

Morseův kód

- První kód s proměnnou délkou kódových značek.
 - Morseovka se používala pro svou technickou nenáročnost a snadné použití na celém světě k vysílání a příjmu zpráv telegrafem (elektrický signál po drátech), nebo radiotelegrafem (přenos radiovými vlnami).
 - Do roku 1996 používána jako prostředek tísňového volání lodí dopravou (nahrazeno satelitními systémy a globálním námořním a tísňovým systémem GMDSS).
 - Dnes používána zejména radioamatéry nebo skauty.
- Morse intuitivně přiřadil krátké kódy v anglickém textu se často vyskytujícím znakům.
- Tři významové kombinace: tečka (dot), čárka (dash), pauza



International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

A	• —	U	• • —
B	• • • •	V	• • — •
C	• — • —	W	• — • —
D	• — • •	X	• • • —
E	•	Y	• • — •
F	• • — •	Z	• — • •
G	• — • —		
H	• • • •		
I	• •		
J	• — • —		
K	• — • —		
L	• • • •		
M	• — • —		
N	• — • •		
O	• — • —		
P	• — • •		
Q	• — • —		
R	• • — •		
S	• • • •		
T	• —		

1	• — • — • —
2	• • • — • —
3	• • — • — •
4	• • • • — •
5	• • • • •
6	• — • • • •
7	• — • — • •
8	• — • — • •
9	• — • — • •
0	• — • — • —

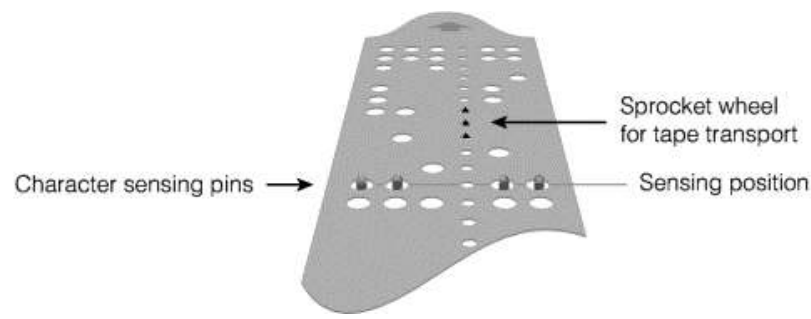
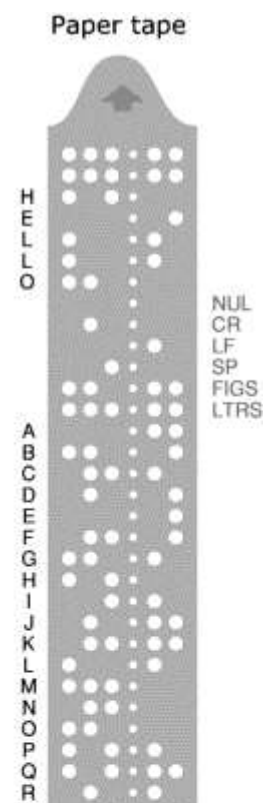
Baudotův kód

- 5-bitový Baudotův kód (ITA 1 – International Telegraph Alphabet No. 1, 1870), Baudot-Murray kód (ITA 2 standard, 1924) je kód, který byl používán pro telegrafní komunikaci až do příchodu ASCII kódu.
 - Běžný text obsahuje cca 50 různých znaků (26 písmen, 10 číslic, 10 interpunkčních znamének, ...).
 - Kód používá 5 bitů na znak, ale kóduje více než 32 znaků. Aby bylo možné pomocí 5 bitů zakódovat takové množství znaků, zavádí se dva kódy se speciálním významem, které slouží k přepínání mezi dvěmi sadami – LETTERS (písmena) a FIGURES (symboly).
 - Šest kódových kombinací má speciální význam (CR, LF, SP, LETTERS, FIGURE, NUL)

LETTERS FIGURES	CODE ELEMENTS	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	CARRIAGE RETUN	LINE FEED	LETTERS	FIGURES	SPACE	ALL-SPACE NOT IN USE
		–	?	:	WHO ARE YOU	3	%	@	£	8	BELL	(	)	.	,	9	0	1	4	'	5	7	=	2	/	6	+						
	1	●	●		●	●	●				●	●						●		●		●		●	●	●				●	●		
	2	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
	3			●						●		●		●	●	○	●	●		●		●	●	○	○	○	○	○	○	○	○	○	○
	4		●	●	●		●	●		●	●			●	●	●			●			●		●	●		●	●	●	●	●	●	●
	5	●						●	●				●	●	●	●	●	●			●	●	●	●	●	●	●	●					

The International Telegraph Alphabet 2

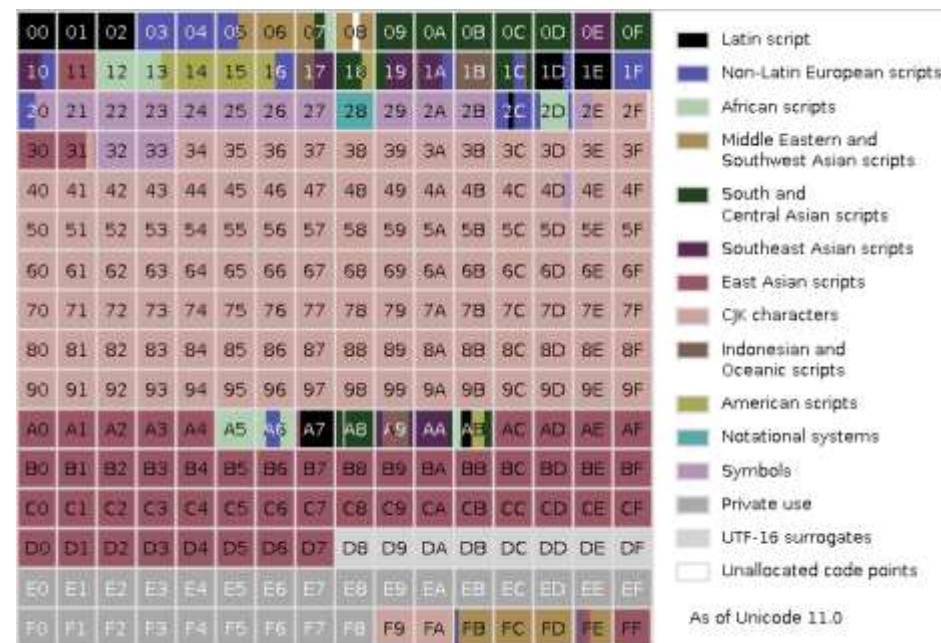
● INDICATES A MARK ELEMENT (A HOLE PUNCHED IN THE TAPE)
○ INDICATES POSITION OF A SPROCKET HOLE IN THE TAPE



Unicode kódování

- *Unicode* je navrženo tak, aby pojmuo více než milion znaků a symbolů a odstraňuje tak limit ASCII v podobě 256 znaků.
- Unicode v.13 (2020) využívá kolem 144 tisíc znaků pokrývajících 154 moderních a historických písem a mnoho sad symbolů (emotikony, apod.)
- Unicode prostor (rozsah 0x0 až 0x10FFFF) je rozdělen na bloky označované jako *plane* po 65536 symbolech.
- Plane 0 označovaný jako Basic Multilingual Plane obsahuje znaky různých moderních i historických písem.
 - Nejvíce prostoru zaujímá čínština, japonština a korejšina (CJK).
- Speciální znak 0xFEFF (Byte Order Mark) uvedený na začátku souboru značí přítomnost Unicode kódování.

Mapa Basic Multilingual Plane (plane 0)
rozdělená po úsecích obsahujících 256 symbolů:



en.wikipedia.org/wiki/Unicode

Unicode kódování

- Existuje několik různých způsobů, jak kódovat řetězce složené ze znaků Unicode.
 - Unicode Transformation Format (UTF) :
UTF-7, UTF-EBCDIC, UTF-8, UTF-16, UTF-32
 - Universal Coded Character Set (UCS):
UCS-2, UCS-4
- Typicky se používá UTF-8 nebo UTF-16. V obou případech se jedná o *kód s proměnnou délkou značek*.
 - UTF-8 je 8-bitové kódování, které je schopno reprezentovat všech 1 112 064 hodnot, navržené tak, aby se maximalizovala kompatibilita s ASCII.
 - Prvních 128 znaků vyskytujících se nejčastěji je kódováno pomocí jednoho bytu, v ostatních případech mohou být použity až čtyři byty.
 - Všechny kódy v Plane 0 jsou přístupné pomocí jednoho 16-bitového slova v UTF-16. Pro ostatní případy se použijí dvě 16-bitová slova.

Unicode kódování

■ Kódování UTF-8 a jeho vlastnosti:

- první byte (tzv. leading byte) indikuje, kolik následuje bytů (tzv. continuation)
- LB nesdílí hodnoty s ostatními byty (LB začíná na 0, 110, 1110 nebo 11110, zatímco ostatní byty vždy na 10). Začátek bytu může být vždy nalezen v maximálně třech čteních.
- Kód je prefixový a tudíž znak je možné dekodovat aniž bychom museli načíst první byte dalšího znaku.
- Kódování respektuje lexikografické pořadí, pokud je provedeme po bytech.
- Kódování dovoluje automatickou autodetekci mezi extended ASCII a UTF-8.

Number of bytes	Bits for code point	First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
1	7	U+0000	U+007F	0xxxxxxx			
2	11	U+0080	U+07FF	110xxxxx	10xxxxxx		
3	16	U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
4	21	U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Character		Binary code point	Binary UTF-8
\$	U+0024	010 0100	00100100
¢	U+00A2	000 1010 0010	11000010 10100010

Metody intuitivní komprese dat

- Komprese textu s omezeným množstvím symbolů
- Komprese slovníkových dat

Komprese textu s omezeným množstvím symbolů (intuitivní metoda)

- Vstupní tok: text s nejvýše 40 různými znaky (26 písmen, 10 číslic, mezera a tři interpunkční znaménka)
- Princip: tok můžeme rozdělit na trojice znaků a každou trojici kódovat pomocí 16 bitů.
 - Protože každý znak nabývá jednu ze 40 hodnot, trojice znaků může nabývat $40^3 = 64000$ hodnot.
 - Protože $40^3 < 2^{16}$, může se tato hodnota vyjádřit pomocí 16 bitů.
- Analýza efektivity:
 - Bez komprese vyžaduje každý ze 40 znaků jeden byte, takže naše intuitivní metoda dává kompresní faktor $3/2 = 1,5$
- Vlastnosti:
 - kompresní faktor konstantní nezávislý na datech
 - výhoda: rozumná míra komprese
 - nevýhoda: metoda není aplikovatelná na ASCII text

Komprese lexikograficky seřazených dat

- Slovníková data (nebo jakýkoliv seznam seřazený lexikograficky) mohou být komprimována pomocí principu *komprese začátků (front compression)*.
 - sousední slova mají tendenci sdílet některé z počátečních znaků.
- Princip: slovo lze komprimovat vynecháním n znaků, které sdílí se svým předchůdcem a jejich náhradou symbolem n .
- Vlastnosti:
 - výhoda: vysoká míra komprese
 - nevýhoda: pro dekódování jednoho slova je nutné procházet seznam od počátku (tj. asymetrická kompresní metoda)

1	a
2	aardvark
3	aback
4	abaft
5	abandon
6	abandoning
7	abasement
8	abandonment
9	abash
10	abated
11	abbot
12	abbey
13	abbreviating
14	abbreviate
15	abbreviation

Dekomprese slova na ř. 14: vezme se šest znaků z předchozího a hledáme číslo menší než 3, to je na ř. 11, dále hledáme číslo menší než 2, viz ř. 3 a následně přejdeme na ř. 1

Komprese lexikograficky seřazených dat

■ Kompresor

```
def compress(dict_orig):  
    wp, dict_comp = '', []  
  
    for w in dict_orig:  
        # nalezení nejdelšího  
        # společného prefixu  
        i = 0  
        while (i < len(wp)) and  
              (i < len(w)) and  
              (w[i] == wp[i]):  
            i += 1  
  
        wp = w  
  
        dict_comp.append((i, w[i:]))  
  
    return dict_comp
```

Pozn: v kódu neřešíme uložení (kódování)

■ Dekomprese jednoho slova na pozici idx

```
def decompress(dict_comp, idx):  
    jp, w = dict_comp[idx]  
  
    for i in range(idx, -1, -1):  
        ip, iw = dict_comp[i]  
        if (ip < jp):  
            w = iw[:jp-ip] + w  
            jp = ip  
  
    return w
```

[(0, 'a'), (1, 'ardvark'), (1, 'back'), (3, 'ft'), (3, 'ndon'), (7, 'ing'), (3, 'sement'), (3, 'ndonment'), (3, 'sh'), (3, 'ted'), (2, 'bot'), (3, 'ey'), (3, 'reviating'), (9, 'e'), (9, 'ion')]



Run-Length encoding (RLE)

- Kódování délek sledů

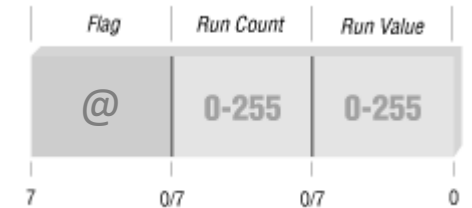
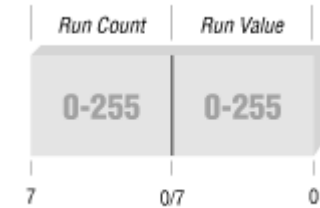
Kódování délkou sledů (Run-length Encoding)

- RLE (Run Length Encoding) je bezeztrátová kompresní metoda, která je založena na nahrazování posloupnosti stejných symbolů jediným kódovým slovem, které obsahuje kód opakujícího se symbolu, počet jeho opakování a identifikátor opakování.
 - Příklad: řetězec AAAAAAAAAAB může být nahrazen kratším řetězcem r9AB, kde r reprezentuje symbol **RLE značky**, 9 udává počet opakování a A je opakovaný symbol.
- Vlastnosti:
 - jednoduchá metoda, lze snadno implementovat v hardware
 - univerzální metoda, lze použít pro kompresi textu i pro kompresi obrazů (po transformaci nebo omezeným počtem barev).
 - metoda využívá znalosti kontextu zpracovávaných symbolů (*kontextová metoda*).
- Účinnost komprese:
 - silně závislá na charakteru vstupních dat. RLE je velmi efektivní, pokud data obsahují sledy stejných symbolů (typické pro černobílé obrázky), a neefektivní pokud se po sobě jdoucí symboly liší.
- Navzdory jednoduchosti se jedná o metodu, která byla a je často používána v praxi
 - komprese binárního obrazu (FAX)
 - komprese obrazových dat s paletou ve formátu BMP, PCX, ...
 - komprese speciálních dat (např. kvantizované koeficienty získané aplikací DCT ve formátu JPEG)



Způsob kódování sledů hodnot

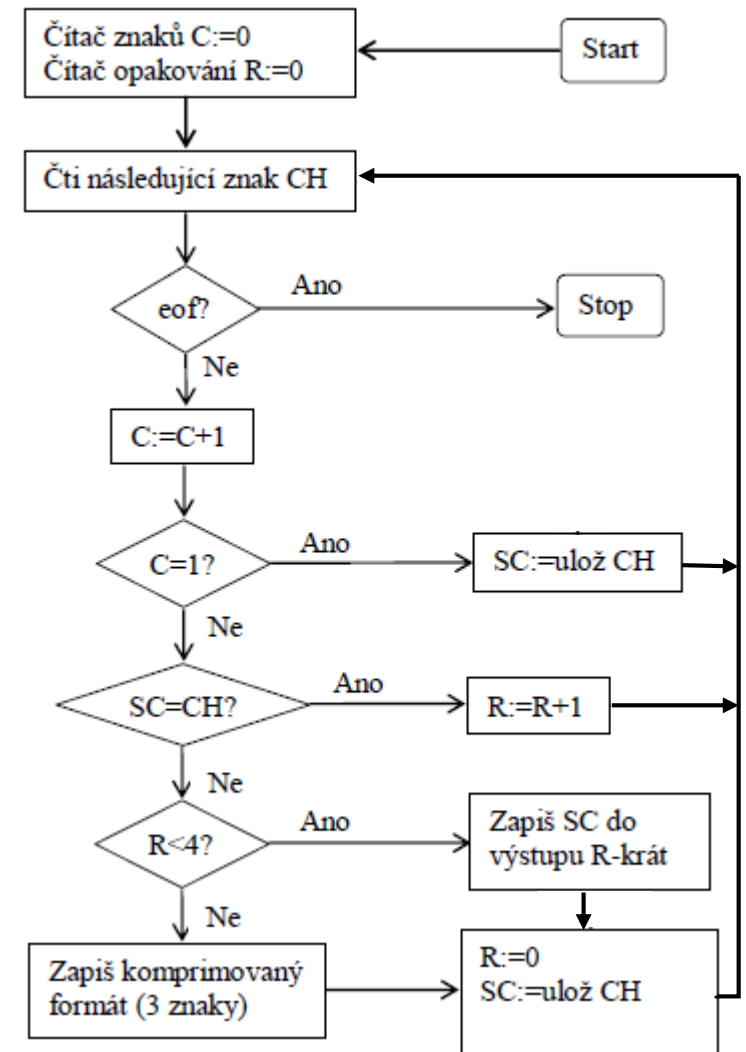
- **Kódování sledů dvojic (počet opakování, opakující se znak)**
 - Typicky se jedná o dvě 8-bitové hodnoty
 - Řetězec AAAAAAAAAAB povede na dvě dvojice: (9, A) (1, B)
 - Řetězec ABABABABAB povede na 10 dvojic (1,A) (1,B) (1,A) (1,B) ...
- **V praxi se využívá kódování využívající tzv. RLE značky (RLE flag)**
 - Implementace: tři pole pro opakující se znaky (RLE značka, počet opakování, opakující se znak) nebo jedno pole s neopakujícím se znakem
 - Řetězec AAAAAAAAAAB povede na trojici (@, 9, A) následovanou B
 - Řetězec ABABABABAB bude kopírován na výstup beze změny



Základní varianta RLE

- Základní varianta, která používá trojici: (RLE značka, počet opakování, opakující se znak)
- Kompresor:

```
def rle(src):
    if not src: return ''
    prev = src[0]
    counter = 1
    result = ''
    for ch in src[1:]:
        if ch == prev:
            counter += 1
        else:
            result += encode(counter, prev)
            prev = ch
            counter = 1
    result += encode(counter, prev)
    return result
```



Základní varianta RLE

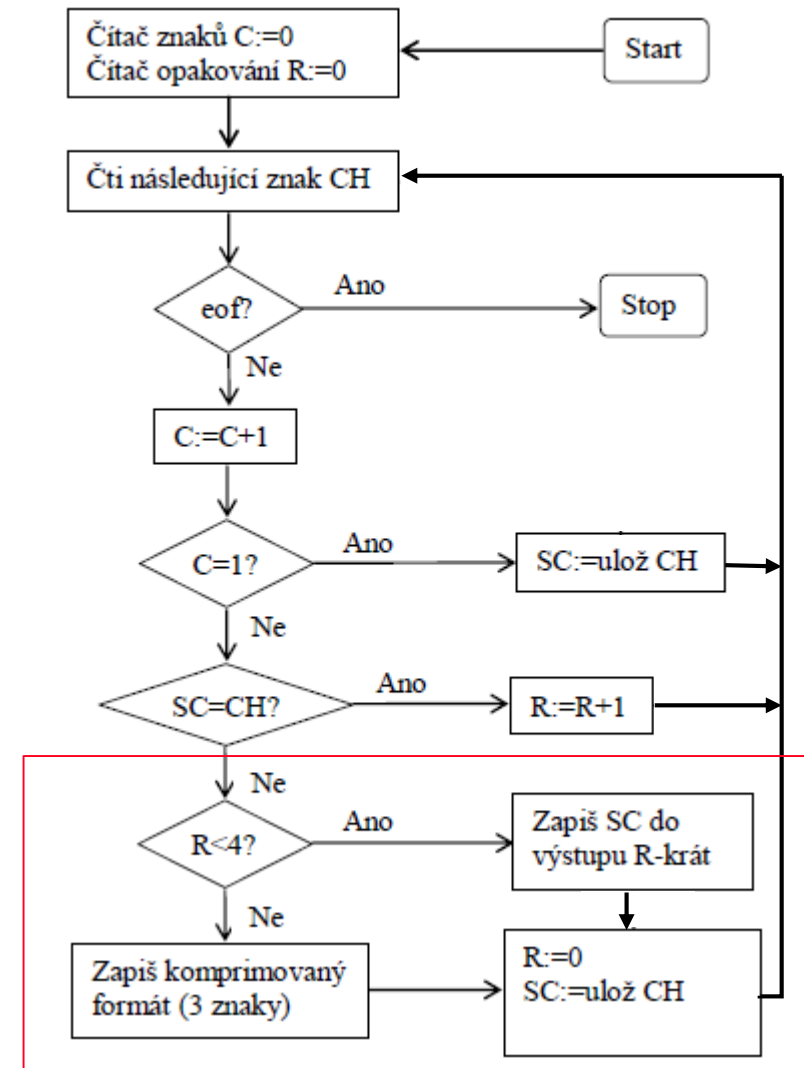
- Základní varianta, která používá trojici: (RLE značka, počet opakování, opakující se znak)

- Enkodér:

- protože nahrazujeme sled třemi symboly, má smysl překódovat pouze sledy délky alespoň čtyři a více.

```
def encode(count, char, rlemarker='@'):
    result = ''
    if (count < 4):
        #write char counter times
        result = char * count
    else:
        #write three chars
        result += rlemarker
        result += chr(count)
        result += char
    return result
```

- Příklad komprese:
Sekvence **AAAAAhojjj** se zkomprimuje
na sekvenci **@\x05Ahojjj**



Základní varianta RLE

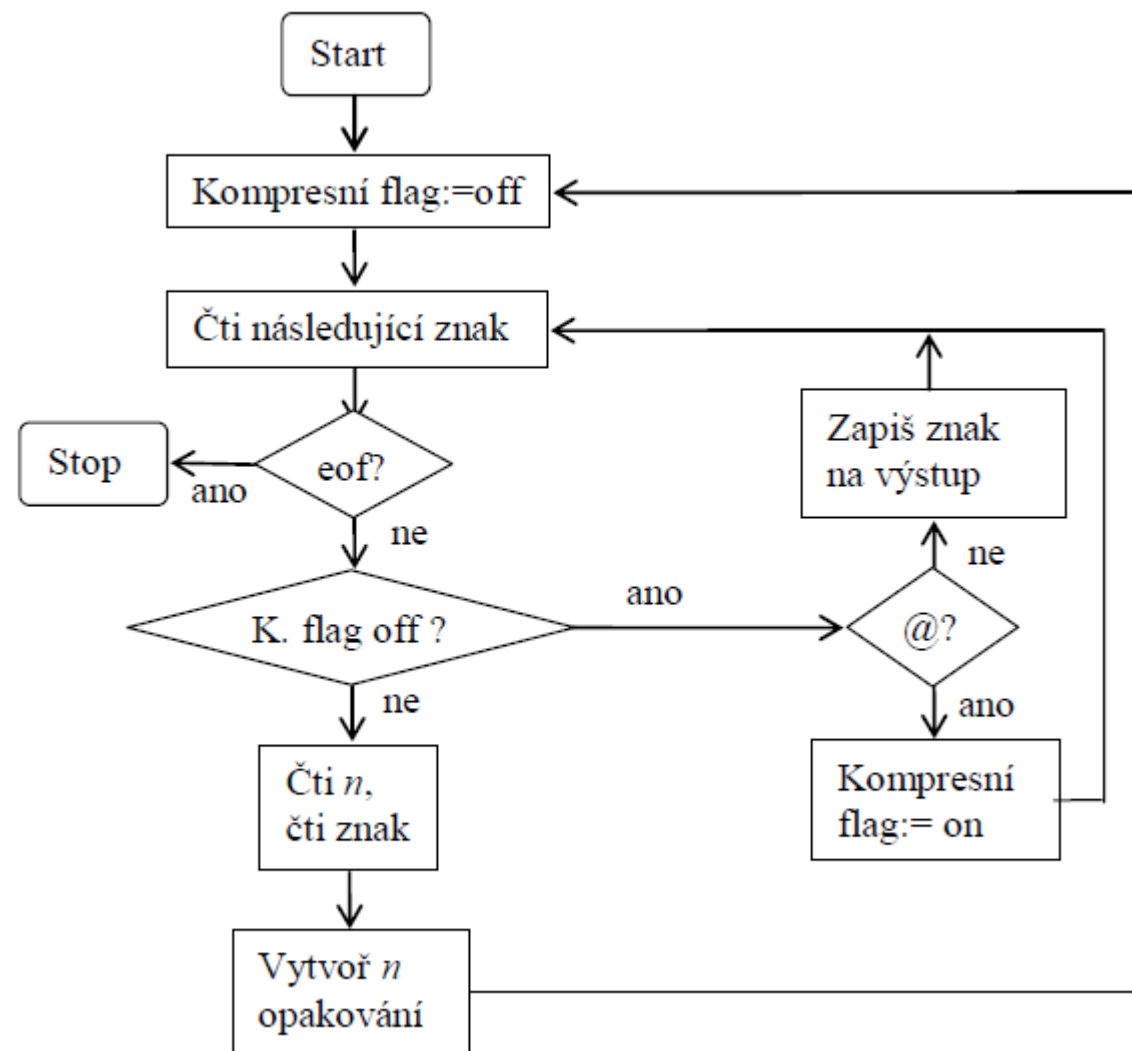
- Základní varianta, která používá trojici: (RLE značka, počet opakování, opakující se znak)
- Dekompresor (optimalizovaná verze):

```
def irle(src, rlemarker='@'):
    if not src: return ''

    flag = False
    counter = 1
    result = ''
    for ch in src:
        if not flag: #read symbol or marker
            if ch == rlemarker:
                flag = True
                continue
        else: #read count
            flag = False
            counter = ord(ch)
            continue

        result += ch*counter
        counter = 1

    return result
```



Základní varianta RLE – odvození kompresní výkonnosti

- Abychom získali představu o kompresním poměru RLE, předpokládejme řetězec N znaků, který se má komprimovat.
 - Předpokládejme, že řetězec obsahuje M opakování s průměrnou délkou L .
 - Každé z M opakování se nahradí 3 znaky (RLE značka, počet a znak), takže velikost komprimovaného řetězce je $N - ML + 3M = N - M(L - 3)$.

- Kompresní faktor je

$$\text{compression factor} = \frac{N}{N - M(L - 3)}$$

- Příklad:

- $N = 1000$, $M = 10$, $L = 4$ dá kompresní faktor $1000/[1000 - 10(4 - 3)] = 1,01$
- Lepší výsledek se získá v případě, kdy $N = 1000$, $M = 50$, $L = 10$, kdy kompresní faktor je $1000/[1000 - 50(10 - 3)] = 1,538$.

Problémy základní varianty RLE

- [illegible]

Problémy základní varianty RLE

- Obecnější druhá varianta vyžaduje změnit enkodér i dekompresor
 - protože komprimovaný symbol může být stejný jako rlemarker, nelze sloučit stav 2 a stav 0
 - úprava enkodéru je triviální

```
def irle(src, rlemarker='@'):
    if not src: return ''
    state, counter, result = 0, 1, ''
    for ch in src:
        if not state:
            if ch == rlemarker:
                state = 1
                continue
            elif state==1: #count
                state = 2
                counter = ord(ch)
                continue
            else: #symbol
                state = 0
                result += ch*counter
                counter = 1
    return result
```

```
def encode(count, char, rlemarker='@'):
    result = ''
    if (count < 4) and (char != rlemarker):
        #write char counter times
        result = char * count
    else:
        #write three chars
        result += rlemarker
        result += chr(count)
        result += char
    return result
```

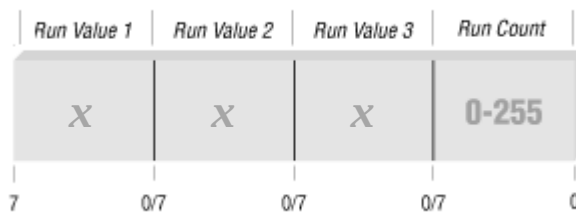

Problémy základní varianty RLE

- Další problém způsobuje zápis počtu opakování jako jeden byte, což znamená, že počet opakování nesmí přesáhnout hodnotu 255
 1. Jednou možností je kódovat sledy po maximálně 255 znacích.
 2. Jinou možností je využít vícestupňové kódování, např. přiřadit nevyužité kombinaci (sled délky 0) speciální význam, kdy další byte určuje počet opakování zmenšený o 256
- První variantu lze vyřešit pouhou změnou enkodéru, dekompresor zůstává bezezměny

```
def encode(count, char, rlemarker='@'):  
    result = ''  
    if (count < 4) and (char != rlemarker):  
        #write char counter times  
        result = char * count  
    else:  
        while count > 0:  
            #write three chars  
            result += rlemarker  
            result += chr( min(count, 255) )  
            result += char  
            count -= 255  
    return result
```

Modifikace základní varianty RLE

- U základní varianty RLE je počet opakování odlišen speciálním symbolem (RLE značkou).
- Mimo to existují i další modifikace lišící se použitým způsobem kódování:
 1. Metoda bez explicitní RLE značky (viz. MNP-5 Microcom).
 2. RLE s více typy značek (jedné pro opakující se symboly a druhé pro sekvence různých symbolů nebo bitového flagu k rozlišení kódovaných symbolů)
 3. RLE pro kompresi sledů určitých hodnot (viz. RLE-0)
 4. Kontextově podmíněné RLE, hierarchické RLE, ztrátové RLE



Metoda RLE bez explicitní RLE značky (použito v protokolu MNP-5 Microcom)

- Jinou možností, jak vyřešit problém s RLE značkou, je využít následujícího principu:
 - Když se ve vstupním toku nachází tři nebo více stejných bytů, kompresor zapíše tři kopie bytu do výstupního toku, následované počtem opakování.
 - Jakmile dekompresor načte tři stejné byty po sobě, očekává automaticky v dalším bytu počet opakování.
- Účinnost:
 - Protože každý sled je nahrazen čtyřmi byty, je kompresní faktor roven
$$\text{compression factor} = \frac{N}{N - M(L - 4)}$$
- Vlastnosti:
 - nevýhoda metody je, že sled délky 3 je reprezentován pomocí 4 znaků.
- Tato metoda byla použita v modemech, kde umožňovala dosáhnout v nejlepším případě komprese 2:1. Průměrně poskytovala kompresi 1.6:1 na textu, tzn. modem 2400 bit/s se tvářil jako by pracoval na přenosové rychlosti ~4000 bit/s.

Metoda RLE bez explicitní RLE značky (použito v protokolu MNP-5 Microcom)

■ Příklady:

- sekvence **AAAAAhojjj** je zkomprimována algoritmem na sekvenci **AAA\x02hojjj**
 - oproti základní variantě RLE dosahujeme úsporu jeden byte
- sekvence **AAAAAhojjjj** je zkomprimována algoritmem na sekvenci **AAA\x02hojjjj\x00**
 - oproti základní variantě dosahujeme úsporu dva byty
- v případě sekvence **AAA** však dochází k expanzi na **AAA\x00**

■ Odpovídající modifikace enkodéru:

```
def encode(count, char):  
    result = ''  
    if (count < 3):  
        #write char counter times  
        result = char * count  
    else:  
        #write two chars  
        result = char * 3  
        result += chr(count-3)  
  
    return result
```

Metoda RLE bez explicitní RLE značky (použito v protokolu MNP-5 Microcom)

- Vzhledem k tomu, že se jednalo o standard, existovaly různé čipy pro podporu HW komprese.

- XR-2403 Enhanced MNP-5 modem Microcontroller
- XR-2443 V.42bis MNP-5 modem Microcontroller

- MNP bylo nahrazeno s příchodem standardu V.42 v té době revoluční slovníkovou metodou BTLZ (British Telecom LZ), která umožňuje dosáhnout komprese až 4:1.

XR-2443 Modem Microcontroller with V.42bis

GENERAL DESCRIPTION

The XR-2443 is a dedicated microcontroller that provides command control for the XR-2400 V.22bis modem chip set. The XR-2443 provides control for CCITT recommended V.42 error correction, including LAPM and MNP 2-4 protocols, with V.42bis BTLZ / MNP 5 data compression. Also supported is the complete AT command set and registers used to control these functions.

The system architecture of the XR-2443 allows the actual command sets for the 'AT', MNP, LAPM and V.42bis to reside external to the XR-2443, allowing ease of customization. Exar provides these command sets to use as is, or the customer can modify to the requirements of the design.

The XR-2443 operates from a single +5 volt power supply, offering low power consumption through CMOS technology.

FEATURES *

V.22bis/V.22/Bell 212A /Bell 103 Modem
Error Free Data Transfer: DATA Mode

- LAPM
- MNP 2-4

MNP Class 5 Data Compression

- 4800 BPS Throughput

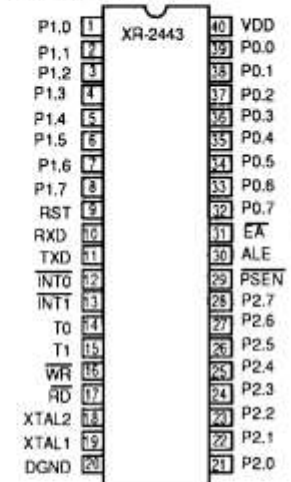
Increased Data Throughput by V.42bis Data Compression

- 9600 BPS Throughput

'AT' Command Control

- Easily Modified, Exar Supplied
- 'AT'/MNP/V.42/V.42bis

PIN ASSIGNMENT



(For other pin assignment diagrams, refer to the end of this datasheet)

ORDERING INFORMATION

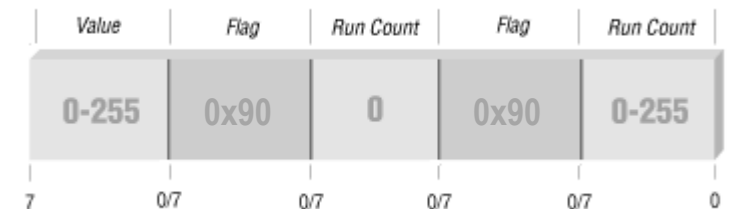
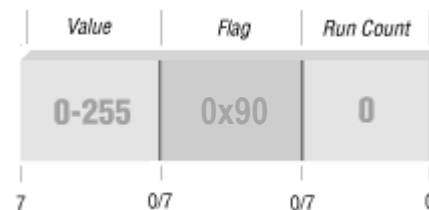
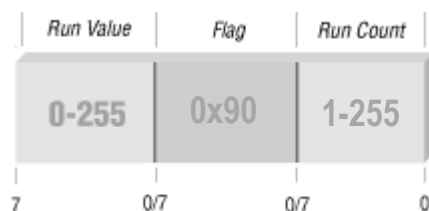
Part Number	Package	Operating Temperature
XR-2443CP	40 Pin Plastic Dip	0°C to 70°C
XR-2443CJ	44 Pin PLCC	0°C to 70°C
XR-2443CQ	44 pin QFP	0°C to 70°C

ABSOLUTE MAXIMUM RATINGS

Power Supply	-0.3V to + 7V
Input Voltage	-0.7V to (VDD +0.3V)
DC Input Current (any input)	±10mA
Power Dissipation (Package Limitation)	1W
Derate above 25°C	11 mW/°C
Storage Temperature Range	-65°C to +150°C

Použití RLE ve formátu BinHex 4.0

- Formát dat BinHex 4.0 (1991) byl navržen a používán jako standard pro kódování binárních ASCII souborů v Macintosh. Pro zamezení chyb při přenosu dat v důsledku nekompatibilit 7-bitových a 8-bitových systémů používá pouze 64 tisknutelných znaků.
- Princip převodu binárního souboru:
 - V první fázi se používá RLE (značka: znak 0x90) a nahrazují se sekvence stejných symbolů.
 - V další fázi se zkomprimovaný binární řetězec kóduje do tisknutelných znaků a přidává se hlavička s CRC.
- Stejně jako u základní varianty se sekvence kratší než čtyři znaky kódují po znacích.
- BinHex zavádí následující pravidla pro zvýšení efektivity RLE:
 1. Opakující se znak se zapisuje jako první, poté následuje RLE značka
 2. Pokud je počet opakování roven nula, pak se nejedná o opakování předchozího znaku, ale o znak 0x90.
 3. Pokud za sekvencí 0x90 0x00 následuje 0x90, jedná o sekvenci opakujících se znaků 0x90.



Použití RLE ve formátu BinHex 4.0

■ Příklady:

- Sekvence tří hodnot 0xFF 0x90 0x04 znamená opakovat znak 0xFF čtyřikrát, tj. 0xFF 0xFF 0xFF 0xFF
 - V tomto případě dosahujeme stejné efektivity jako základní varianta RLE, která použije také 3 byty.
- Komprimovaná sekvence 0x1B 0x90 0x00 kóduje dva symboly a sice 0x1B 0x90
 - V tomto případě máme úsporu 1 byte oproti základní variantě RLE, která musí vygenerovat pro 0x90 tři byty
- Sekvence pěti hodnot 0x1B 0x90 0x00 0x90 0x05 odpovídá 0x1B 0x90 0x90 0x90 0x90 0x90
 - V tomto případě přidáváme o jeden byte navíc oproti základní variantě RLE, která opakující se znaky 0x90 zakóduje pomocí trojice

■ Odpovídající modifikace enkodéru:

```
def encode(count, char, rlemarker='\x90'):
    result = ''
    if (count < 4) and (char != rlemarker):
        #write char counter times
        result = char * count
    else:
        result += char
        if (char != rlemarker):
            #write two chars
            result += rlemarker
            result += chr(count)
        else:
            #write two or three chars
            result += '\x00'
            if (count > 1):
                result += rlemarker
                result += chr(count)
    return result
```

Použití RLE ve formátu BinHex 4.0

■ Komprese

```
import binhex, string, base64

# zakódování řetězce do binhex4
open('test.txt', 'w').write('AAAAhojj')
binhex.binhex('test.txt', 'test.hqx')
```



■ obsah souboru test.hqx:

```
(This file must be converted with BinHex 4.0)

: #(4PFh3ZG(Kd!&4&@&3rN!3!N!8*!*!%F9""N!9SEftU
YVX!!!:
```

■ odpovídající binární reprezentace:

```
'\x08test.txt\x00TEXT?\x90\x04\x00\x90\x05\t
\x00\x90\x04qPA\x90\x05hojj\xbb\x00\x00'
```

■ Převod na binární reprezentaci

```
# načtení dat, kontrola hlavičky
s = open('test.hqx').read().splitlines()
assert 'BinHex 4.0' in s[0] and s[2][0] == ':' and s[-1][-1] == ':'
s = ''.join(s[2:])[1:-1]

binhex4_chars = '!"#$%&\ '()*+,-012345689@ABCDEFGHIJKLMNPQRSTUVWXYZ`_abcdefghijklmnopqrstuvwxyz0123456789+/'
base64_chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'
while len(s) % 4: s += '=' #padding

# využijeme existence funkce na dekódování BASE64
c = s.translate(string.maketrans(binhex4_chars, base64_chars))
print repr(base64.b64decode(c))
```



RLE pro kompresi sledů určitého symbolu (RLE-0)

- V jistých případech se vyskytují často sledy jediného symbolu, které bychom rádi kódovali efektivněji (např. nuly v případě Move-to-Front transformace, černé pixely v obraze, apod.). K tomuto účelu se používá např. *Wheelerův 1/2 kód*.
- Předpokládejme, že chceme kódovat sledy symbolu 0 pomocí *Wheelerova 1/2 kódu*
 - Vstupní abeceda se rozšíří o jeden nový symbol Θ .
 - Kombinaci Θ a 0 použijeme tak, abychom zakódovali sled symbolů 0 následovně:
První symbol 0 nebo Θ kóduje 1 nebo 2 po sobě jdoucí symboly 0.
Následující symbol 0 nebo Θ kóduje 2 nebo 4 po sobě jdoucí 0. Třetí 4 nebo 8, čtvrtý 8 nebo 16 atd.
 - Kódové slovo pro sled délky n lze získat tak, že vezmeme binární reprezentaci hodnoty $n + 1$, vynecháme nejvíce významový bit, místo bitu nula uložíme symbol 0 a místo bitu jedna symbol Θ .
- Příklady:
 - Sekvence A0B se zakóduje jako A0B, sekvence A00B se zakóduje jako A Θ B.
 - Dále A000B se zakóduje jako A00B, a A0000B pomocí A Θ 0B
 - Sled délky sedm A0000000B se kóduje pomocí tří symbolů A000B

RLE pro kompresi sledů určitého symbolu (RLE-0)

- Důležitou vlastností Wheelerova 1/2 kódu je, že použité kódování nikdy nezpůsobí expanzi.
 - Izolovaný symbol 0 se zakóduje jako jeden symbol.
 - Sledy o alespoň dvou symbolech se zakódují vždy úsporněji (viz tabulka).
- Nevýhodou je však nutnost rozšířit vstupní abecedu o jeden symbol, což může vnést jistou režii při kódování ostatních symbolů.
- Tato kompresní metoda se někdy označuje jako RLE-0 (Zero Run Length Encoding)

Vstupní řetězec	Délka sledu n	Wheelerův kód	$n + 1$ binárně
A0B	1	0	01
A00B	2	00	11
A000B	3	000	001
A0000B	4	0000	101
A00000B	5	00000	011
A000000B	6	000000	111
A0000000B	7	0000000	0001

Komprese obrazových dat pomocí RLE (obecný princip)

- **RLE je přirozený kandidát pro kompresi grafických dat**
 - pokud zvolíme jeden pixel obrazu, pak je velká šance, že sousední pixely budou mít stejnou barvu
 - princip lokality lze využít nejen pro 2D data (komprese rastrového obrazu), ale také pro 3D (komprese volumetrických dat, hierarchické RLE)
- **Podle počtu bitů (bitových rovin) rozlišujeme:**
 - Komprese binárních obrazů (black&white, bitmap)
 - Komprese obrazů v odstínech šedi (typicky 8-bit grayscale)
 - Komprese bitmap s více bitovými rovinami (4-bit, 8-bit, 16-bit, 24-bit)
- **Grafická data jsou dvojrozměrná**
 - různé možnosti serializace dat (horizontální, vertikální, zig-zag, blokové, ...)
 - využití replikace o obou směrech (horizontální i vertikální)

Komprese binárních obrazových dat pomocí RLE

■ RLE kodér

```
def rle(pixels):
    result = []
    current = 1
    counter = 0
    for pixel in pixels:
        pixel = pixel & 1
        if pixel == current:
            counter += 1
        else:
            result.append(counter)
            counter = 1
            current = pixel

    result.append(counter)
    return result
```

■ RLE dekodér

```
def irle(src):
    current = 1
    result = []
    length = 0
    for i in src:
        for j in range(i):
            result.append(current)
            length += 1
        current = current ^ 1
    return result
```

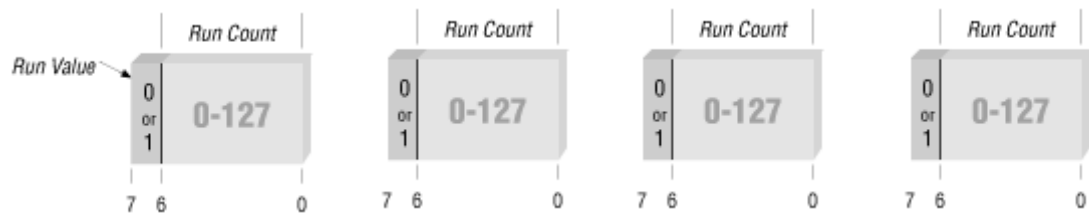
```
from PIL import Image
im = Image.open("test.png")
im_bw = im.convert('1') # convert image to black and white
c = rle(im_bw.getdata()) # get data and compress
d = irle(c) # decompress
im_bw.putdata(d) # put data back
im_bw.save("test-reco.png")
```

Kódování počtu opakování

- Možná řešení pro případy, kdy počet opakování přesáhne hodnotu 255:
 - Zapsat do výstupu hodnotu 255 následovanou hodnotou 0; stejným mechanismem ošetřit zápis počtu opakování zmenšeného o 255

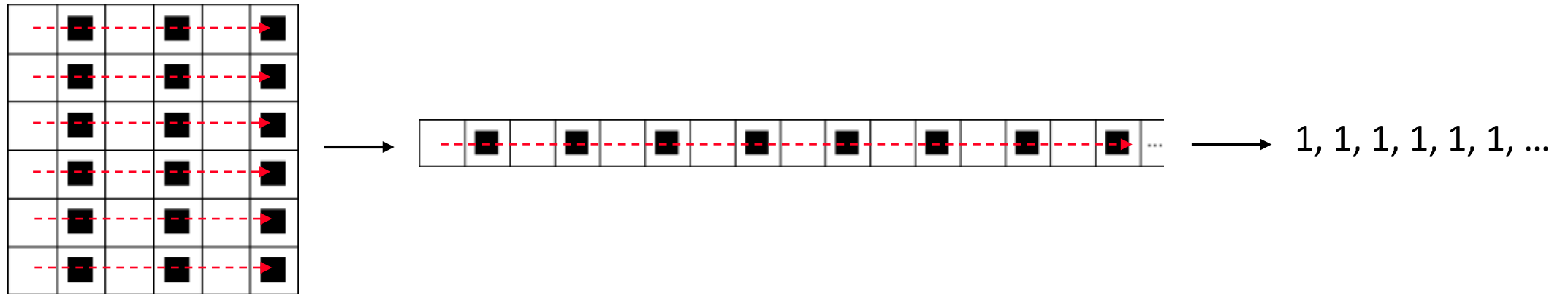
```
def encode(count):  
    result = ''  
    while count > 0:  
        result += chr( min(count, 255) )  
        result += '\\0'  
        count -= 255  
    return result
```

- Alokovat jeden bit pro určení opakující se hodnoty (tzv. bit-level RLE paket)



Komprese binárních obrazových dat pomocí RLE

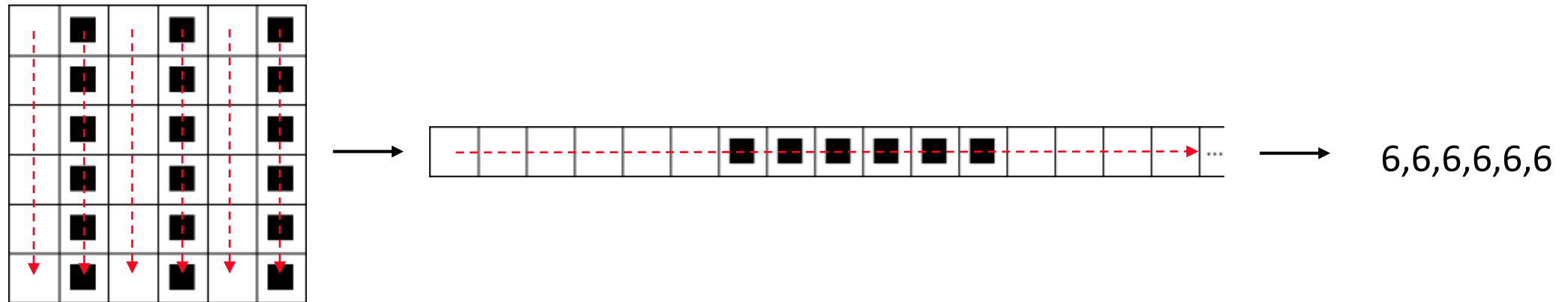
- Velikost komprimovaného toku závisí na složitosti obrazu.
 - Čím více obsahuje detailů, tím je komprese horší.
 - Nejhorší případ nastane, pokud se budou neustále střídát hodnoty 0 a 1. Poté dochází k výrazné expanzi, neboť každá změna způsobí uložení hodnoty 1.



- Pokud např. použijeme prosté 8-bitové binární kódování, dojde k osminásobné expanzi.

Komprese binárních obrazových dat pomocí RLE

- Abychom co nejvíce eliminovali případy, kdy dochází k expanzi v důsledku častých změn v obraze, provádí se skenování bitmapy typicky ve více směrech.



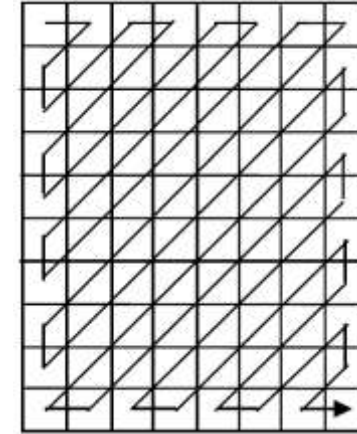
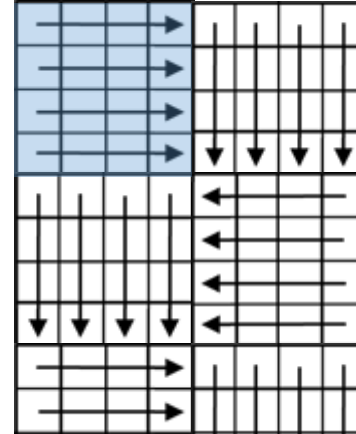
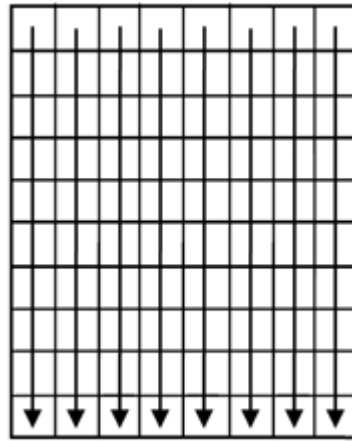
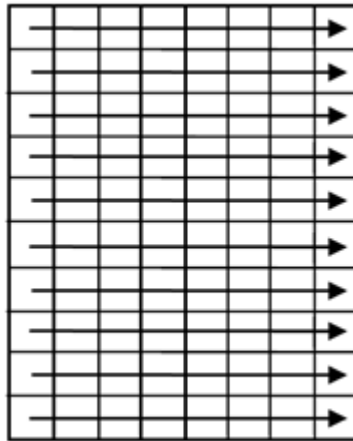
- Implementace kompresoru

- Dobrý a praktický RLE kompresor obrazů by měl být schopen skenovat bitmapu více způsoby a následně vybrat a do výstupního souboru uložit tu variantu, která dává nejvyšší kompresní poměr.
- Výhodné bývá také rozdělit obraz na několik částí a ty skenovat nezávisle.

Metody skenování 2D dat (serializace)

■ Základní přístupy

- lineární formy, dlaždicové zpracování, zigzag forma



■ Další formy

- Bustrofédon (boustrophedonic), perimeter, Morton, Hilbert, chevron, ...

Komprese binárních obrazových dat (ITU-T T4 standard pro FAX, 1999)

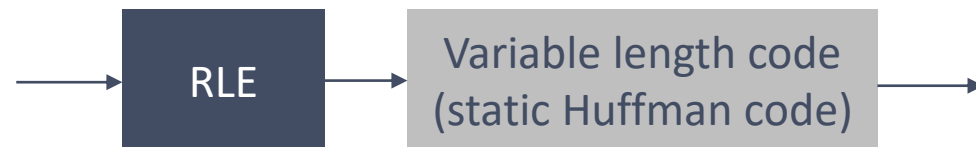
- Účinnost komprese se dá dále zvýšit, pokud zkombinujeme RLE s entropickým kódem jako tomu je např. ve standardu ITU-T T4 pro kódování binárních dat při přenosu obrazu pomocí FAXu.

- Princip komprese:

- Na začátek každého řádku je uměle vložen bílý pixel (hodnota 1).

Sekvence pixelů 00001100000 tedy bude pomocí RLE převedena na sekvenci čtyř hodnot: 1,4,2,5.

- Výstup RLE zmenšený o jedna je kódován pomocí statického Huffmanova kódu, jehož kódová slova byla získána na základě analýzy mnoha běžně přenášených dokumentů.



- Huffmanův kód:

- Kódy jsou definovány zvlášť pro bílé a černé pixely.
- Kódy jsou definovány až pro sledy délky 64 (hodnota 63). Delší sledy jsou kódovány pomocí dvou speciálních kódových značek, které mají význam příčti 64 a příčti 128.

Příklad: délka 150 se zakóduje pomocí kódu 128+ následovaného kódem pro 22.

run-length	white codeword	black codeword
0	00110101	0000110111
1	000111	010
2	0111	11
3	1000	10
4	1011	011
..		
20	0001000	00001101000
..		
64+	11011	0000001111
128+	10010	000011001000

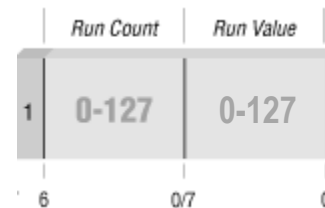
Příklad: sekvence z příkladu (1,4,2,5) je zakódována pomocí 19 bitů: 00110101,10, 000111,011

Komprese obrazových dat ve stupních šedi (obecný princip)

- RLE se dá použít i pro kompresi bitmapy ve stupních šedi. Každý sled pixelů stejné intenzity (stupně šedi) se kóduje jako dvojice (délka sledu, hodnota pixelu). Na délku sledu je vyhrazen obvykle jeden byte, což umožňuje popsat sledy délky až 255 pixelů.
 - **Příklad:** Bitmapa s 8-bitovou stupnicí šedi, která začíná 12, 12, 12, 12, 12, 12, 12, 12, 12, 35, 76, 112, 67, 87, 87, 87, 5, 5, 5, 5, 5, 5, 1, ... se komprimuje na sekvenci **9**, 12, 35, 76, 112, 67, **3**, 87, **6**, 5, 1, ... , kde zvýrazněné číslo definuje počet opakování následující hodnoty.

Možnosti kódování:

1. Je-li obraz omezen na 128 stupňů šedi, můžeme obětovat jeden bit v každém bytu na indikaci, zda byte obsahuje hodnotu šedé, nebo počet.
Příklad: Sekvence z ukázky se uloží jako: **0x89**, 12, 35, 75, 112, 67, **0x83**, 87, **0x86**, 5, 1, ..., přičemž je-li nastaven nejvyšší bit (hodnota > 127), tak byte kóduje počet opakování.

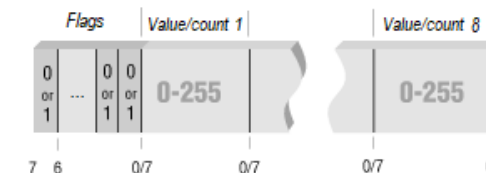


2. Je-li počet stupňů šedi 256 a můžeme jej redukovat na 255, pak lze hodnotu 255 použít jako RLE značku informující o tom, že následující byte udává počet opakování
Příklad: Sekvence z ukázky se uloží jako: **255**, 9, 12, 35, 76, 112, 67, **255**, 3, 87, 255, 6, 5, 1, ...

Komprese obrazových dat ve stupních šedi (obecný princip)

■ Další možnosti:

3. K rozlišení typu bytu můžeme použít jeden bit. Bity můžeme skládat do skupin po osmi a každá skupina se zapíše do výstupního toku před nebo za 8 bytů, ke kterým patří.



Celkový počet přídavných bytů je ovšem $1/8$ velikosti výstupního toku (obsahují jeden bit pro každý byte výstupního toku), takže zvětšují velikost tohoto toku o 12,5%.

Příklad: Sekvence z ukázky se uloží jako $10000010_2, 9, 12, 35, 76, 112, 67, 3, 87, 100 \dots_2, 6, 5, 1, \dots$,

4. Můžeme použít RLE s dvěma typy značek, kdy skupina n různých pixelů se zakóduje jako sekvence začínající hodnotou $-n$ následovanou hodnotami n pixelů.

Tímto však redukuje maximální délku sledu na 128 opakování.

Protože délka sledu nemůže být nula, můžeme zapisovat počet opakování snížený o jedna. V nejhorším případě obraz může obsahovat opakující se sekvenci hodnot p_1, p_2, p_2 .

Tato je zakódována jako čtveřice $-1, p_1, 2, p_2$ (alternativně $-3, p_1, p_2, p_2$) a dochází u 8-bit obrazu k expanzi o 33%.



Příklad: Sekvence z ukázky se uloží jako $9, 12, -4, 35, 76, 112, 67, 3, 87, 6, 5, X, 1, \dots$, kde hodnota X je kladná nebo záporná dle toho, jaká hodnota následuje za 1.

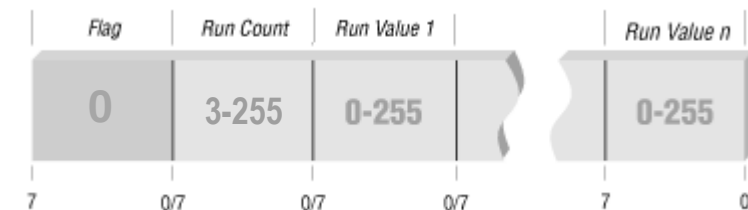
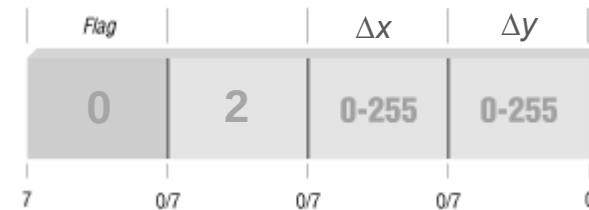
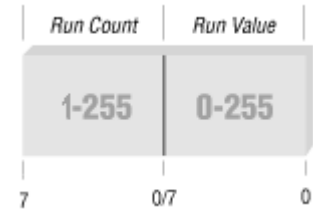
Použití metody RLE ve formátu BMP

- BMP (1988) je jeden ze základních univerzálních grafických formátů, který se používá pro ukládání rastrové grafiky a není zatížen patentem či licencí.
- Formát BMP
 - Umožňuje reprezentovat obrazy monochromatické i barevné na *různé hloubce barev*. Jeden pixel může být reprezentován jedním až 24 bity dle požadovaného počtu barev: 2 barvy (1 bit na pixel), 16 (4 bity), 256 (8 bitů), 65 536 (16 bitů), nebo 16,7 miliónů barev (24 bitů).
 - Osmibitové obrázky mohou místo barev používat šedou škálu (256 odstínů šedi) nebo barevnou paletu (Color LookUp Table).
 - Je podporována komprese a alfa kanály.
 - Datová struktura souboru: hlavička se signaturou **BM**, hlavička s informacemi o datech a způsobu uložení, barevná paleta, rastrová data
- BMP používá jednoduchý způsob RLE komprese obrazů se 4 nebo 8 bitovými plochami (RLE4 – 4 bity na pixel RLE8 – 8 bitů na pixel).

Použití metody RLE ve formátu BMP

RLE8

- U obrazů s osmi bitovými plochami jsou pixely pomocí RLE kódované do dvojic bytů (C,P). První byte dvojice *C* udává počet opakování, druhý byte *P* je hodnota pixelu, která se opakuje *C* krát.
 - Příklad: dvojice 0x04 0x02 se expanduje na čtyři pixely 0x02 0x02 0x02 0x02.
- Hodnota *C*=0 má speciální význam, který je určen hodnotou *P*:
 - Kombinace *C*=0 a *P*=0 značí konec řádku. Zbývající pixely jsou doplněny hodnotou 0x00.
 - Kombinace *C*=0 a *P*=1 značí konec obrazu. Zbytek obrazu je vyplněn hodnotou 0x00.
 - Kombinace *C*=0 a *P*=2 informuje dekodér o tom, že má přejít na jinou pozici v obrazu. Pozice je určena dvěma následujícími byty, které udávají počet sloupců a řádků, které se mají přeskočit, abychom došli k nenulovému pixelu.
 - Kombinace *C*=0 a *P*>2 značí, že následuje *P* neopakujících se pixelů, které jsou zakódované v následujících *P* bytech.



Použití metody RLE ve formátu BMP

RLE8 - Příklady

- Posloupnost bytů: `0x08 0x00`
se expanduje na hodnoty pixelů:

```
00 00 00 00 00 00 00 00
```

- Zakódovaná posloupnost: `0x04 0x15 0x00 0x00 0x02 0x11 0x02 0x03 0x00 0x01`
se expanduje na dva řádky pixelů doplněné zprava a zespodu nulami dle velikosti obrazu:

```
15 15 15 15 00 ... ..  
11 11 03 03 00 ... ..  
... ..
```

- Předpokládáme-li obraz 8x4 8-bitových pixelů, pak následující posloupnost:

```
0x04 0x02, 0x00 0x04 0xA3 0x5b 0x12 0x47, 0x01 0xf5, 0x02 0xe7, 0x00 0x02 0x00 0x01,  
0x01 0x99, 0x03 0xc1, 0x00 0x00, 0x00 0x04 0x08 0x92 0x6b 0xd7, 0x00 0x01
```

je reprezentace 32 pixelů:

```
02 02 02 02 a3 5b 12 47  
f5 e7 e7 00 00 00 00 00  
00 00 99 c1 c1 c1 00 00  
08 92 6b d7 00 00 00 00
```

(C, P)

00 00	EndOfLine
00 01	EndofImage
00 02 XX YY	Skip X cols Y rows
00 XX	pixels

Použití metody RLE ve formátu BMP

RLE4

- Obrazy se čtyřmi bitovými plochami se komprimují podobným způsobem. Dvojice bytů (C, P) však reprezentuje byte počtu a byte dvou čtyřbitových hodnot pixelů, které se střídají.
 - Příklad `0x05 0xA2` je komprimovaná reprezentace pěti 4-bitových pixelů `A 2 A 2 A`, zatímco dvojice `0x07 0xFF` reprezentuje sérii sedmi čtyřbitových pixelů `F F F F F F F`. Zakódovaná dvojice `0x05 0x56` se expanduje na posloupnost 4-bitových hodnot: `5 6 5 6 5`.
- Mimo to v případě dvojice $(0, C)$, kde $C > 2$, následuje C čtyřbitových hodnot, sbalených po dvojicích do jednoho bytu. Hodnota C je normálně násobek 4, což předpokládá, že dvojice $(0, C)$ specifikuje pixely pro vyplnění celočíselný počet slov (kde slovo jsou dva byty). Když C není násobek 4, pak se zbytek posledního slova vyplní nulami.
 - Příklad: `0x00 0x08 0xA3 0x5B 0x12 0x47` specifikuje 8 pixelů `A 3 5 B 1 2 4 7`
`0x00 0x06 0xA3 0x5B 0x12` udává šest pixelů a plní čtyři byty hodnotami `A 3 5 B 1 2 0 0`

Další varianty RLE

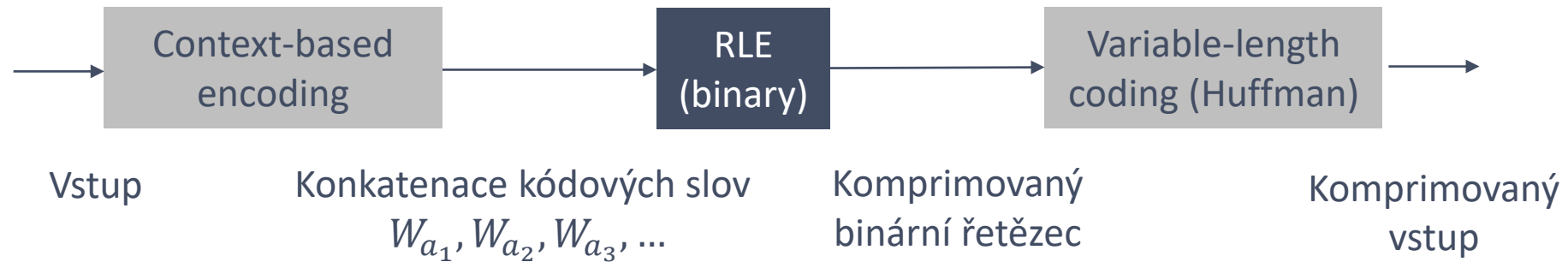
Ztrátová komprese obrazů

- Pokud se budou ignorovat krátké sledy, lze dosáhnout ještě lepších kompresních poměrů.
 - Při kompresi obrazů dochází k ztrátě informace, ale někdy je to pro uživatele přijatelné (významné příklady, kdy nejsou přijatelné žádné ztráty, jsou rentgenové snímky a astronomické snímky pořízené velkými teleskopy, kde je cena snímku astronomická)
- Parametr: jaká největší délka sledu se ještě smí zanedbat.
 - Když uživatel určí např. 3, pak se sloučí všechny sledy délky 1, 2 nebo 3 s jejich sousedy. V tomto případě by se výstupní sekvence z RLE ve tvaru 6,8,1,2,4,3,11,2 uložila jako 6,8,7,16, kde 7 je součet $1+2+4$ (sdružení tří sledů) a 16 je součet $3+11+2$.
- Tento způsob komprese má smysl u obrazů s vysokým rozlišením, kdy ztráta nějakého detailu může být okem nepostřehnutelná, ale může se tak výrazně redukovat velikost výstupního toku.

Další varianty RLE

Komprese obrazu pomocí kontextově podmíněného RLE kódování (conditioned RLE)

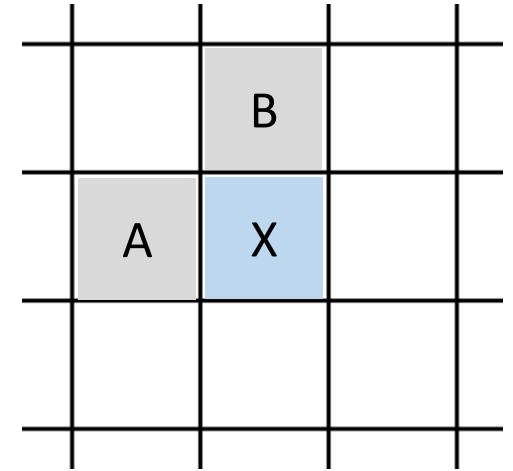
- Další modifikace RLE (Gharavi 1987) spočívá v zavedení kódování před RLE, které má usnadnit následnou kompresi.



- Pro obraz s n stupni šedi metoda v první fázi přiřadí každému pixelu n -bitový kód, který je zvolen dle hodnot pixelu a jeho blízkých sousedů tak, aby po spojení všech n -bitových slov vznikl ideálně řetězec obsahující sledy nul a jedniček. Na binární řetězec je v další fázi aplikováno RLE jehož výstup je kódován kódem s proměnnou délkou.
- Metoda využívá znalosti kontextu (kontext druhého řádu).

Komprese obrazu pomocí kontextově podmíněného RLE kódování

- Metoda považuje každý rozkladový řádek obrazu za Markovský model druhého řádu.
 - hodnota dat (X) závisí právě na dvou předchozích sousedech (A, B)
- Pro sadu trénovacích obrazů se pro každou dvojici hodnot sousedů A, B spočítá, kolikrát jednotlivé hodnoty X nastávají.
 - Pokud mají A a B stejnou hodnotu, je přirozené předpokládat, že i X má tuto hodnotu.
 - Když mají A a B velmi rozdílné hodnoty očekáváme, že i X může nabývat mnoha různých hodnot, a to s nízkou pravděpodobností.
- Počty pak poskytují podmíněné pravděpodobnosti $P(X|A, B)$, tj. pravděpodobnost, že aktuální pixel bude mít hodnotu X , pokud okolní pixely mají hodnoty A, B .



Pixely použité k predikci X

Komprese obrazu pomocí kontextově podmíněného RLE kódování

- Každému pixelu v komprimovaném obrazu se přiřadí nový čtyřbitový kód a to v závislosti na jeho podmíněné pravděpodobnosti.
 - Příklad: Předpokládejme aktuální pixel X s hodnotou 1, jehož sousedé mají hodnoty $A=3$, $B=1$. Tabulka ukazuje, že podmíněná pravděpodobnost $P(1|3, 1)$ je vysoká. X se přiřadí nový 4-bitový kód **s málo sledy nul a jedniček**. Naopak stejnému $X=1$ se sousedy $A=3$ a $B=3$ se může přiřadit nový 4-bitový kód s mnoha sledy, protože tabulka udává, že podmíněná pravděpodobnost $P(1|3, 3)$ je nízká.
- část výsledků získaných čtením trénovacích obrazů s 4-bitovými pixely:

hodnota X		přiřazené kódové slovo							
A	B	W1	W2	W3	W4	W5	W6	W7	...
2	15	hodnota: 4	3	10	0	6	8	1	...
		počet: 21	6	5	4	2	2	1	...
3	0	hodnota: 0	1	3	2	11	4	15	...
		počet: 443	114	75	64	56	19	12	...
3	1	hodnota: 1	2	3	4	0	5	6	...
		počet: 1139	817	522	75	55	20	8	...
3	2	hodnota: 2	3	1	4	5	6	0	...
		počet: 7902	4636	426	264	64	18	6	...
3	3	hodnota: 3	2	4	5	1	6	7	...
		počet: 33927	2869	2511	138	93	51	18	...
3	4	hodnota: 4	3	5	2	6	7	1	...
		počet: 2859	2442	240	231	53	31	13	...

Komprese obrazu pomocí kontextově podmíněného RLE kódování

- Konstrukce kódu s určitým počtem sledů.
 - Vygenerujeme-li všech šestnáct 4-bitových kódů W_1 až W_{16} zjistíme, že dva kódy, 0000 a 1111 mají jeden sled, a 0101 a 1010 mají čtyři sledy. Kódy se tedy musí uspořádat v pořadí rostoucího počtu sledů. Pro 4-bitové kódy dostaneme čtyři skupiny
- Kódy skupiny i mají i sledů. Kódy jsou v každé skupině vybrány tak, že kódy ve druhé polovině skupiny jsou komplementem kódů z první poloviny v obráceném pořadí.
- Když se vybere kód pro X , porovná se s předcházejícím kódem. Je-li nejnižší významový bit předcházejícího kódu 1, aktuální kód se invertuje. Předpokládá se, že se takto sníží počet sledů.
- Všechny 4-bitové kódy uspořádané do čtyř skupin dle počtu sledů
 - W_1 až W_2 : 0000, 1111
 - W_3 až W_8 : 0001, 0011, 0111, 1110, 1100, 1000,
 - W_9 až W_{14} : 0100, 0010, 0110, 1011, 1101, 1001,
 - W_{15} až W_{16} : 0101, 1010
- Konstrukce 5-bitových kódů
 - W_1 až W_2 : 00000, 11111
 - W_3 až W_{10} : 00001, 00011, 00111, 01111, 11110, 11100, 11000, 10000,
 - W_{11} až W_{22} : 00010, 00100, 01000, 00110, 01100, 01110, 11101, 11011, 10111, 11001, 10011, 10001
 - W_{23} až W_{30} : 01011, 10110, 01101, 11010, 10100, 01001, 10010, 00101,
 - W_{31} až W_{32} : 01010, 10101

Souhrn

- Principem RLE je reprezentovat subsekvence identických symbolů nazývané sled (angl. run) pomocí dvojice (L, s) , kde L udává počet opakování symbolu s .
- V případě binárních řetězců (binární data, případně obraz) není nutné kódovat s , neboť hodnota bitu se pravidelně střídá mezi 0 a 1.
- Pokud máme data, kde se vyskytují často sledy jednoho symbolu, je výhodné použít kompresní metodu RLE-0 (Zero Run Length Encoding), která vede vždy k redukci.

TRANSFORMACE

- Smyslem transformace je převést problém do reprezentace, která je s ohledem na další zpracování výhodnější.
 - Příklady: diskretní Fourierova transformace (DFT), diskretní kosinová transformace (DCT)
- Transformace typicky zachovává množství informace. Z pohledu komprese tedy nedochází k redukci množství dat. Ke kompresi může dojít až ve spojení s další technikou.
 - DCT se na signál dívá jako na směsici kosinusovek s různou frekvencí, což nám umožňuje provést kompresi snížením úrovně detailů (např. odstranění složek, které nejsou slyšitelné).
 - Burrows-Wheeler transformace aplikovaná na textový řetězec zvyšuje účinnost komprese pomocí RLE.
- V následující části se podíváme na dvě transformace, které lze použít pro bezeztrátovou kompresi.
 - *Move-to-Front transformace (MTF)*
 - *Burrows-Wheeler transformace (BWT)*

Move-to-front transformace (MTF)

■ *Move-to-Front transformace*

- Ryabko, 1980, původně “book stack”
- cílem je získat sekvenci malých 8-bitových hodnot (ne nutně stejných)

■ Princip

- Udržovat *seznam naposledy použitých symbolů* (abecedu A) vyskytujících se v komprimovaném toku ve formě seřazené posloupnosti a kódovat symboly jako index do této posloupnosti (do abecedy).
- Index lze chápat též jako počet symbolů, které aktuálnímu v abecedě předchází.
- Po zpracování symbolu se symbol přesune z aktuální pozice v posloupnosti na její začátek.

- Příklad pro vstup BACACAAAAD a abecedu $A=\{A,B,C,D\}$

vstup (iterace)	sekvence	seznam
BACACAAAAD	inicializace	A,B,C,D
B ACACAAAAD	1	B,A,C,D
B A CACAAAAD	1,1	A,B,C,D
BAC C ACAAAAD	1,1,2	C,A,B,D
BAC A CACAAAAD	1,1,2,1	A,C,B,D
BACAC C AAAAD	1,1,2,1,1	C,A,B,D
BACAC A AAAAD	1,1,2,1,1,1	A,C,B,D
BACACAA A AD	1,1,2,1,1,1,0	A,C,B,D
BACACAA A AD	1,1,2,1,1,1,0,0	A,C,B,D
BACACAAA A D	1,1,2,1,1,1,0,0,0	A,C,B,D
BACACAAA D	1,1,2,1,1,1,0,0,0,3	D,A,C,B

Vlastnosti Move-to-front transformace

- Transformace převádí vstupní sekvenci symbolů na sekvenci čísel, která mají *nízké hodnoty* a v případě opakujících se symbolů či dokonce skupin symbolů produkuje sekvenci stejných hodnot.
 - Příklad: Vstup *ABCDEFGHJKLMNOPQRSTUVAABABABABABABABAACABABEFEFEFEF* tvořený 22 různými symboly vede na sekvenci
0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,20,21,0,1,1,1,1,1,1,1,1,1,1,1,0,21,1,2,1,1,20,20,1,1,1,1,1,1
 - Příklad: Vstup *ABCDEFGHABCDEFGHABCD* obsahující osm různých symbolů vede na sekvenci
0,1,2,3,4,5,6,7,7,7,7,7,7,7,5,5,5,5
- Transformace je *reverzibilní*.
- Jedná se o lokálně adaptivní metodu, která dává dobré výsledky pokud vstupní tok má tzv. *koncentrační vlastnost*, tj. když se lokální četnosti výskytu symbolů výrazně liší oblast od oblasti (angl. local frequency correlation).
- V kombinaci s dalšími technikami a kódováním umožňuje komprimovat.

Move-to-front transformace (implementace)

■ MTF transformace

```
def MTF(text, dictionary=None):
    # Initialise the list of characters
    if not dictionary: dictionary =
        [chr(i) for i in range(0,255)]

    result = []
    for c in text:
        # find the rank of the character
        # in the dictionary
        rank = dictionary.index(c)
        result.append(rank)

        # update the dictionary
        dictionary.pop(rank)
        dictionary.insert(0, c)

    return result
```

■ Zpětná MTF transformace

```
def iMTF(sequence, dictionary=None):
    # Initialise the list of characters
    if not dictionary: dictionary =
        [chr(i) for i in range(0,255)]

    result = ""
    for rank in sequence:
        # read the rank of the character
        # from dictionary
        result += dictionary[rank]

        # update dictionary
        e = dictionary.pop(rank)
        dictionary.insert(0, e)

    return result
```

- | | | | | | |
|-------|---|----|--|----|------------|
| 1 | MTF('BACACAAAAD') | => | iMTF([66, 66, 67, 1, 1, 1, 0, 0, 0, 68])
'BBC\x01\x01\x01\x00\x00\x00D' | => | BACACAAAAD |
| <hr/> | | | | | |
| 2 | MTF('BACACAAAAD',
sorted(set('BACACAAAAD'))) | => | [1, 1, 2, 1, 1, 1, 0, 0, 0, 3]
iMTf([1, 1, 2, 1, 1, 1, 0, 0, 0, 3],['A','B','C','D']) | => | BACACAAAAD |



Move-to-front transformace (použití pro kompresi dat)

- Kódování po bytech je nevýhodné a nepřineslo by žádnou úsporu. V praxi se MTF používá zejména ve spojení s kódováním s proměnnou délkou značek.



- Možné přístupy:

1. Použít **statický Huffmanův kód** celých čísel v rozsahu $[0, n]$ sestavený tak, že menší celá čísla dostanou kratší kódy. Nevýhody: kód nemusí odpovídat četností, omezení n . Výhoda: jednoduché.
Příklad Huffmanova kódu pro celá čísla od 0 do 7 : 0-0, 1-10, 2-110, 3-1110, 4-11110, 5-111110, 6-1111110, 7-1111111.
2. Použít **jiný typ statického kódování**, např. *gamma kód*. Výhoda: n není shora omezeno
Příklad kódu pro několik čísel: 1-1, 2-010, 3-011, 4-00100, 5-00101, ..., 16-000010000
3. Provést **dva průchody** vstupním tokem. První průchod počítá četnosti výskytu jednotlivých hodnot. V druhém průchodu se provede vlastní kódování, které využívá četnosti k určení pravděpodobností a přiřazení kódu (Huffmanův kód, Aritmetický kód, apod.). Výhoda: kompaktní. Nevýhoda: pomalé.
4. Použít **plně adaptivní metody** kódování (adaptivní Huffmanův kód, apod.)

Eliasovo gama kódování (gamma encoding, γ code)

- Eliasův *gama kód* (Elias, 1975) je univerzální kód pro reprezentaci kladných celých čísel, u kterých není předem známa jejich horní hranice.
- Typicky se využívá ke kompresi dat, ve kterých jsou malé hodnoty mnohem častější než velké (to je případ MTF), neboť nižším hodnotám je přiřazen kratší kód.
- Hodnota i ($i \geq 1$) je v gama kódu reprezentována pomocí $2\lfloor \log_2 i \rfloor + 1$ bitů
- Princip kódování čísla i :
 - Nechť 2^k je nejvyšší mocnina, která je menší nebo rovna i , tzn. $2^k \leq i < 2^{k+1}$, a tedy $k = \lfloor \log_2 i \rfloor$.
 - Na výstup nejprve zapíšeme k nulových bitů.
 - K těm přidáme $k + 1$ bitů, které odpovídají binární reprezentaci i .
- Ekvivalentní způsob:
 - Zakódujeme k pomocí *unárního kódu* jako k nulových bitů následovaných jedničkovým bitem.
 - Následně připojíme k bitů hodnoty i

Number	Binary	γ encoding	Implied probability
$1 = 2^0 + 0$	1	1	1/2
$2 = 2^1 + 0$	1 0	0 1 0	1/8
$3 = 2^1 + 1$	1 1	0 1 1	1/8
$4 = 2^2 + 0$	1 0 0	0 0 1 0 0	1/32
$5 = 2^2 + 1$	1 0 1	0 0 1 0 1	1/32
$6 = 2^2 + 2$	1 1 0	0 0 1 1 0	1/32
$7 = 2^2 + 3$	1 1 1	0 0 1 1 1	1/32

Příklad

- Uvažujme vstupní řetězec $x = \text{'ZEBRU ZEBOU ZUBY BRRRRRR'}$ o délce 24 znaků (192 bitů)
- Sekvence x zvětšená o 1 kódovaná Gamma kódem dává výstup o délce 306 bitů

```
0000001011011 0000001000110 0000001000011 0000001010011 0000001010110 00000100001
0000001011011 0000001000110 0000001000011 0000001010000 0000001010110 00000100001
0000001011011 0000001010110 0000001000011 0000001011010 00000100001 0000001000011
0000001010011 0000001010011 0000001010011 0000001010011 0000001010011 0000001010011
```

- Výstupem MTF je sekvence $y = \text{MTF}(x)$

```
[90, 70, 68, 83, 86, 37, 5, 5, 5, 82, 5, 5, 5, 2, 4, 90, 4, 2, 7, 0, 0, 0, 0, 0]
```

- Sekvence y zvětšená o 1 kódovaná Gamma kódem dává výstup o délce 160 bitů

```
0000001011011 0000001000111 0000001000101 0000001010100 0000001010111 00000100110
00110 00110 00110 0000001010011 00110 00110 00110 011 00101 0000001011011 00101
011 0001000 1 1 1 1 1
```

Move-to-front transformace (použití pro kompresi dat)

- MTF typicky produkuje sekvence obsahující mnoho nul, zejména ve spojení s Burrows-Wheeler transformací, kde sekvence nul mohou tvořit až 50%. V případě, že chceme dosáhnout co nejvyšší míry komprese, je nutné provádět kódování efektivně.
- Pro zvýšení účinnosti komprese se proto mohou kódovat sledy nul pomocí RLE-0 a teprve poté se aplikují další techniky:



- Zavedení RLE je výhodné zejména v případě použití kódování, které neumožňuje reprezentovat jeden bit vstupu méně než jedním bitem (např. Huffman, gama kód). Naopak, RLE není nezbytně nutné v případě použití adaptivního aritmetického kódování.

Varianty MTF

- Dá se ukázat, že metoda MTF pracuje v nejhorším případě trochu hůře než Huffmanovo kódování. V nejlepším případě dá výrazně lepší výsledek.
- Omezením základní varianty MTF je, že přesun na začátek se provádí bez ohledu na četnost výskytu daného symbolu. Mimo to je operace přesunu nesymetrická – přesun na začátek je rychlý a odsun nepotřebných symbolů pomalý.
 - Symbolům, které se vyskytují často mohou být přiřazovány vyšší hodnoty díky tomu, že jsou vytlačovány zřídka kdy se vyskytujícími symboly.
- Varianta *Sticky MTF* přesouvá symbol na začátek ihned, avšak pokud není tentýž symbol použit znovu v dalším kroku, přesune na pozici odpovídající cca 40% původní pozice.
 - Využívá se skutečnosti, že při kompresi textu je více než 60% pravděpodobnost, že symbol, který byl přesun na počátek bude použit znovu.
 - Aby se zamezilo zaplevelení fronty symboly, které se vyskytnou jen jednou, odsouvají se zřídka se vyskytující symboly zpět za aktivní symboly.
- Varianta *Move-ahead-k* provádí přesun symbolu o k pozic směrem dopředu.
 - Za cenu snížení efektivity u vstupních toků majících koncentrační vlastnost se zvyšuje efektivita u případů, kde tato vlastnost neplatí.
 - Pokud $k=|A|$, pak se jedná o základní variantu MTF.
 - Případ, kdy $k=1$, spočívá a prohození pořadí se sousedem.

Vylepšení kompresní účinnosti – varianty MTF

- Varianta *Wait-c-and-move* provádí přesun symbolu $a \in A$ pouze tehdy, pokud se vyskytl alespoň c -krát (nemusí to být souvisle c -krát za sebou).
 - Každý prvek A musí mít svůj čítač pro počítání počtu shod.
 - Tato metoda má smysl u realizací, kde je přesouvání a přeskládání prvků A pomalé.
- Varianta *Move-One-From-Front* (M1FF) provádí přesun symbolu a na pozici 1 tehdy, pokud se aktuálně nachází na pozici 2 a dále. Jinak se provede přesun na pozici 0.
- Varianta *Move-One-From-Front-2* (M1FF2) provádí přesun symbolu a na pozici 1 tehdy, pokud se nachází aktuálně na pozici 2 a dále, a pouze tehdy, pokud symbol na začátku seznamu byl použit v jednom z předchozích dvou kroků. Jinak se provede přesun na pozici 0.
 - Jedná se o drobnou modifikaci M1FF, která však vyžaduje implementovat čítač
- Dalším způsobem, jak ovlivnit účinnost MTF je *redukovat velikosti abecedy*.
 - Výhodou je, že se mohou generovat menší hodnoty. Nevýhodou může být nutnost uložit abecedu do výstupního toku.

Varianty MTF

- Varianty MTF mohou vylepšit kompresní účinnost, avšak jedná se typicky o nízké jednotky procent.
 - Např. Sticky MTF v průměru dosahuje úspory 1%. Může však nastat také expanze, viz případ komprese Excel souboru (exc1), kdy dochází k tomu, že výsledek MTF produkuje vesměs sekvence malých hodnot, avšak s nepravidelně se vyskytujícími úseky hodnot s vysokou variabilitou.

Files from Calgary Corpus															
	bib	book1	book2	geo	news	obj1	obj2	paper1	paper2	pic	progc	progl	progp	trans	Average
Normal	1.943	2.390	2.037	4.482	2.497	3.865	2.456	2.452	2.411	0.779	2.492	1.708	1.699	1.491	2.336
Sticky	1.927	2.357	2.014	4.428	2.465	3.797	2.433	2.440	2.389	0.753	2.478	1.698	1.702	1.489	2.312
Reduction	0.8%	1.4%	1.1%	1.2%	1.3%	1.8%	0.9%	0.5%	0.9%	3.5%	0.6%	0.6%	−0.2%	0.1%	1.0%
Files from small Canterbury Corpus															
	csrc	excl	fax	html	lisp	man	play	poem	sprc	tech	text	Average			
Normal	2.081	0.971	0.779	2.436	2.536	3.108	2.508	2.390	2.608	1.993	2.249	2.151			
Sticky	2.097	1.237	0.753	2.410	2.547	3.098	2.483	2.361	2.571	1.969	2.228	2.159			
Reduction	−0.8%	−22%	3.5%	1.1%	−0.4%	0.3%	1.0%	1.2%	1.4%	1.2%	0.9%	−0.4%			

Burrows-Wheeler transformace

- Transformaci BWT navrhl Wheeler v roce 1983, ale kompresní metoda využívající tento princip byla představena až v roce 1994. BWT podobně jako předchozí metody (RLE, MTF) zohledňuje kontext ale přistupuje k němu poměrně netradičním způsobem. Metoda se nesnadno klasifikuje a typicky se řadí do ostatních metod.
- Na rozdíl od předchozích přístupů se vstup zpracovává po blocích (metoda pracuje v *blokovém režimu*).
- Burrows-Wheeler transformace nemění množství informace obsažené ve vstupním řetězci. Smyslem je **přeuspořádat jednotlivé symboly** vstupního řetězce tak, aby bylo možné jej následně reprezentovat kompaktněji.
- Komprese na bázi BWT je obecně použitelná, dobře funguje pro obrázky, zvuk i text, a může dosahovat velmi vysoké účinnosti (1 bit na byte, nebo i lepší).



Burrows-Wheeler transformace – princip

- Hlavní myšlenka metody BW je **získat** ze vstupního řetězce S tvořeného n symboly jeho **permutaci** $L = \pi(S)$ **splňující dvě podmínky**:

1. V každé části L vznikají koncentrace několika symbolů. Jinak řečeno, že když se symbol s najde v jisté poloze L , pak se další výskyty s budou s velkou pravděpodobností nacházet poblíž.
2. Z L lze rekonstruovat původní řetězec S .



- Důsledky požadovaných vlastností

- První podmínka znamená, že L se dá snadno a efektivně komprimovat metodou MTF nebo RLE. To rovněž znamená, že metoda BW bude dobře fungovat pouze pro **velká n** , typicky několik tisíc symbolů.
- Protože L je permutace S , tak platí, že obsahuje stejný počet symbolů. Abychom mohli rekonstruovat původní řetězec, budeme potřebovat nějakou dodatečnou informaci.

- Počet permutací n symbolů je roven $n!$, což je i pro malé hodnoty obrovské číslo.
 - Cílem není nalézt optimální permutaci, ale získat obecný algoritmus generující vhodnou permutaci.

Burrows-Wheeler transformace – princip

- Autoři BWT ukázali, že vhodnou permutací L je možné získat **rotací řetězce** S následujícím způsobem:
 - Utvoříme množinu řetězců tvořenou cyklickým posuvem řetězce S o nula až $|S| - 1$ symbolů doleva.
 - Řetězce abecedně seřadíme, čímž vznikne lexikograficky seřazená matice M .
 - Permutace L odpovídá poslednímu sloupci matice M .
- Je zřejmé, že tímto způsobem získané L je permutací S .
- BWT přeuspořádá symboly tak, že související symboly jsou sdruženy u sebe (viz dále), čímž je splněna podmínka 1. Otázkou zůstává, jak z L získat původní podobu S .

■ Příklad:

- Pozn: Zarážka $\$$ není v praxi nutná, slouží k ilustraci přístupu.



Burrows-Wheeler transformace – princip

- **L obsahuje koncentrace identických symbolů**
 - po seřazení se za sebou objeví všechny permutace, které mají stejný počáteční znak. Sloupec L obsahuje znak, který předchází, takže L bude obsahovat koncentraci podobných znaků.
- **Příklad:**
 - Předpokládejme, že slova bail, fail, hail, jail, mail, nail, pail, rail, sail, tail, wail se vyskytují někde v S . Po seřazení se u sebe objeví všechny permutace, které začínají na il. Frázi il ale vždy předchází a, takže L bude obsahovat koncentraci a-ček. K sobě se dostanou rovněž všechny permutace, začínající ail, a přispívají ke koncentraci písmen bfhjmnprstw v jedné oblasti L . Podobně i koncentrace i-ček předcházejících l.
- Metodu BW lze charakterizovat jako postup, který pomocí seřazování shlukuje symboly podle jejich kontextů. Metoda však uvažuje kontext pouze na jedné straně každého symbolu.

```

.....
ail.....b
ail.....f
ail.....h
ail.....j
ail.....m
ail.....n
ail.....p
ail.....r

.....
il.....ba
il.....fa
il.....ha
il.....ja
il.....ma
il.....na
.....

```

Burrows-Wheeler transformace – princip

- Ukázka části matice M , která vznikne aplikací BW transformace na Shakespearovo dílo Hamlet.
- $L = \dots nnhn\nnnnnngnnnnnnnnnnjnn\dots$
- Zatímco BWT sama o sobě nepřináší žádnou úsporu, výhoda BWT je patrná ve spojení s další technikou, která těží z výskytu shluků stejných či opakujících se symbolů (např. RLE nebo MTF).

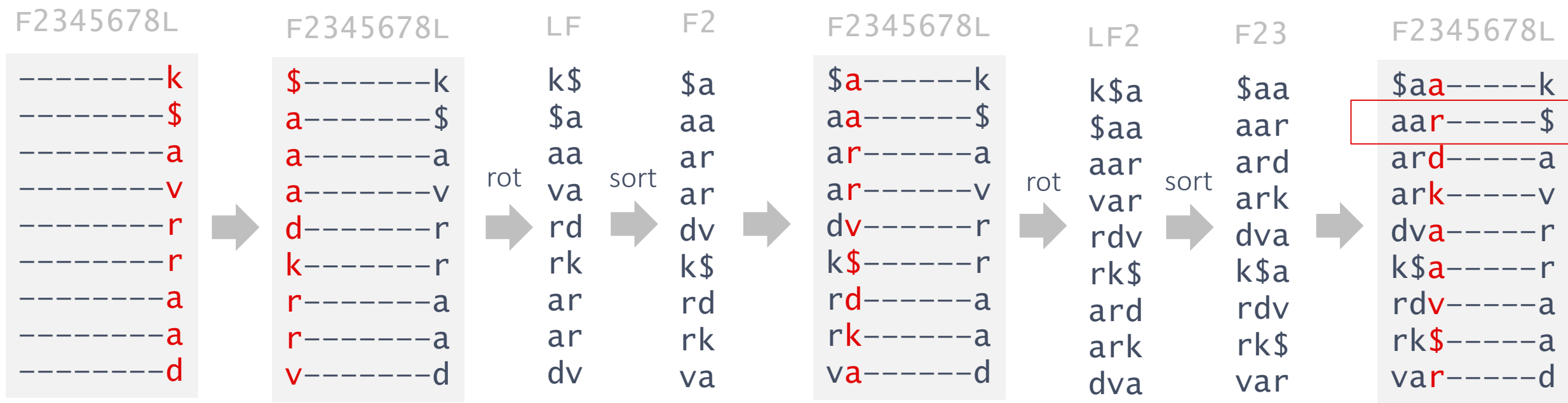
```

ot look upon his like again. ... n
ot look upon me; Lest with th... n
ot love on the wing,-- As I p... h
ot love your father; But that... n
ot made them well, they imita... n
ot madness That I have utter'... n
ot me'? Ros. To think, my lor... n
ot me; no, nor woman neither,... n
ot me? Ham. No, by the rood, ... g
ot mend his pace with beating... n
ot mine own. Besides, to be d... n
ot mine. Ham. No, nor mine no... n
ot mock me, fellow-student. I... n
ot monstrous that this player... n
ot more like. Ham. But where ... n
ot more native to the heart, ... n
ot more ugly to the thing tha... n
ot more, my lord. Ham. Is not... j
ot move thus. Oph. You must s... n
ot much approve me.--well, si... n

```

Burrows-Wheeler transformace – princip zpětné transformace

- Výhodou BW metody je, že existuje zpětná transformace, kterou lze efektivně implementovat. Tím splníme podmínku 2.
 - Zpětná transformace je principiálně založena na postupné rekonstrukci matice M sloupec po sloupci s využitím skutečnosti, že obsah L předchází F .
- Příklad:
 - $L = k\$avrraad$, pokud prvky L seřadíme, dostáváme první sloupec $F = \$aaadkrrv$.
 - Zarážka $\$$ slouží k ilustraci principu.



Burrows-Wheeler transformace – zpětná transformace

- Zpětná transformace založená na kompletní rekonstrukci matice by vedla na paměťově i časově neefektivní řešení.
 - matice má rozměr $n \times n$, tj. n^2 prvků
 - bylo by nutné spojovat a řadit podřetězce
- Pokud přiřadíme jednotlivým symbolům S hodnotu určující kolikrát se daný symbol již vyskytl, můžeme si všimnout, že pořadí symbolů v F i L je stejné.
 - Této vlastnosti lze využít pro konstrukci algoritmu pro efektivní rekonstrukci S .

- Příklad:

S : $a_0 a_1 r_0 d_0 v_0 a_2 r_1 k_0 \$$

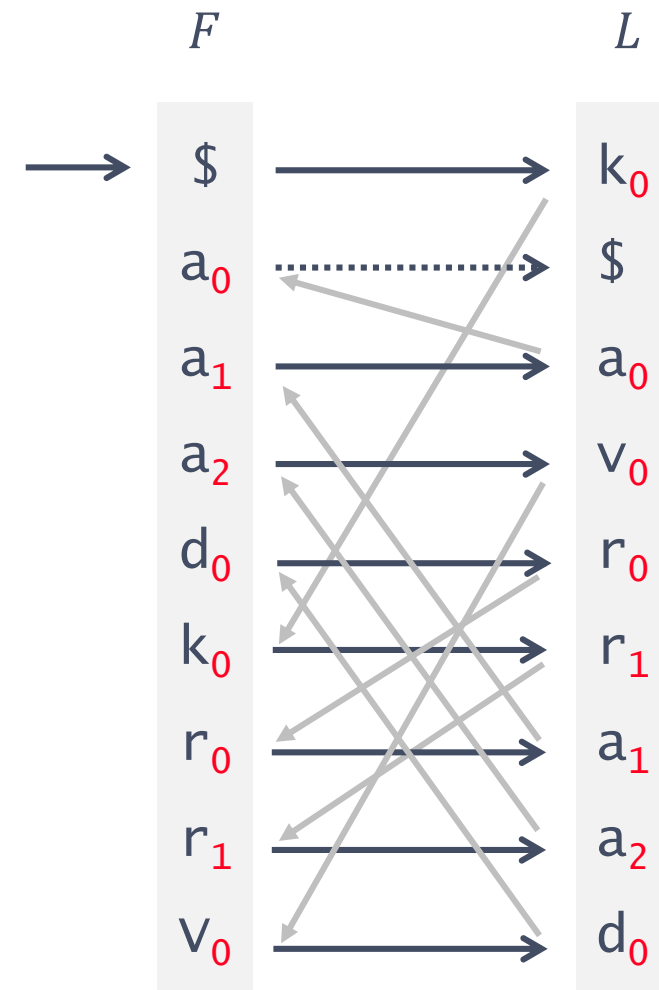
F	L
$\$$	$a_0 a_1 r_0 d_0 v_0 a_2 r_1 k_0$
a_0	$a_1 r_0 d_0 v_0 a_2 r_1 k_0 \$$
a_1	$r_0 d_0 v_0 a_2 r_1 k_0 \$ a_0$
a_2	$r_1 k_0 \$ a_0 a_1 r_0 d_0 v_0$
d_0	$v_0 a_2 r_1 k_0 \$ a_0 a_1 r_0$
k_0	$\$ a_0 a_1 r_0 d_0 v_0 a_2 r_1$
r_0	$d_0 v_0 a_2 r_1 k_0 \$ a_0 a_1$
r_1	$k_0 \$ a_0 a_1 r_0 d_0 v_0 a_2$
v_0	$a_2 r_1 k_0 \$ a_0 a_1 r_0 d_0$

Matice M

Burrows-Wheeler transformace – zpětná transformace

- Zpětná transformace začíná v F výběrem řádku, který obsahuje poslední symbol, tj. \$
- V L nalezneme na stejném řádku symbol k_0 předcházející \$.
- Nalezneme v F řádek obsahující symbol k_0 . Přejdeme na odpovídající řádek v L a získáváme další symbol a sice r_1 .
- Tímto získáme celý řetězec S , avšak pozpátku.

- Příklad:



S : $a_0 a_1 r_0 d_0 v_0 a_2 r_1 k_0 \$$

Burrows-Wheeler transformace (implementace)

■ Transformace BWT

- není-li nutné získat paměťově a časově efektivní řešení, implementace BWT je triviální
- místo zarážky se používá index

```
def BWT(s):  
    n = len(s)  
  
    # compute all cyclic permutations  
    # and sort them in lexicographic order  
    M = sorted([s[i:n]+s[0:i] for i in range(n)])  
  
    # locate the original sequence in M  
    I = M.index(s)  
  
    # get the sequence of last letters  
    L = ''.join([q[-1] for q in M])  
  
    return (I, L)
```

■ Příklad pro vstup THISISHIST

```
n = 10  
M= [ 'HISISHISTT',  
      'HISTTHISIS',  
      'ISHISTTHIS',  
      'ISISHISTTH',  
      'ISTTHISISH',  
      'SHISTTHISI',  
      'SISHISTTHI',  
      'STTHISISHI',  
      'THISISHIST',  
      'TTHISISHIS']  
  
I = 8  
L= 'TSSHIIITS'
```

Burrows-Wheeler transformace (implementační detaily)

- Uvedená implementace dopředné BWT je naivní a paměťově neefektivní.
 - Metoda BWT je účinná hlavně pro dlouhé řetězce (nejméně tisíce symbolů), tzn. praktická implementace musí dovolovat velké hodnoty n , což však vede na expanzi a matice $n \times n$ prvků se nevejde do dostupné paměti.
- Řešení:
 - všechny operace kodéru (vytvoření permutací a jejich seřazení) se provádí v jednorozměrném poli velikosti n . V paměti **stačí mít pouze** původní řetězec S a pomocné pole R stejné velikosti.
 - Získat k -tou permutaci znamená procházet S od k -tého symbolu postupně směrem k n -tému a dále od 0 až po $k - 1$.
 - Porovnání dvou permutací je operace, kterou lze realizovat přímo nad S , jen se liší počáteční indexy.
- Postup:
 1. Nechť S je indexováno od 0 do $n - 1$. Pole R je inicializováno jako $R = [0, 1, \dots, n]$.
 2. Pole R seřadíme podle řetězců začínajících na pozici $S[R[i]]$ a končících na pozici $S[(R[i] + n) \bmod n]$.
 3. Po seřazení odpovídá R prvnímu sloupci matice, tj. F ; i -tou položku sloupce L získáme tak, že přečteme z S symbol, který předchází symbolu na pozici $R[i]$, tj. $L[i] = S[(R[i] - 1) \bmod n]$.

Burrows-Wheeler transformace (implementační detaily)

- Efektivní implementace BWT má lineární paměťovou složitost, tj. $O(n)$.
- Časová složitost je dána zejména náročností řazení (R lze vytvořit v lineárním čase)
 - Dobré řadící algoritmy (např. populární QuickSort) mají průměrnou asymptotickou složitost odpovídající $O(n \log n)$.
 - Existují však případy, kdy složitost QuickSort metody dosahuje $O(n^2)$. To je případ sekvencí, které obsahují sledy stejných symbolů, tj. částečně seřazených dat. Extrémem je kompletně seřazená sekvence.
 - Na výkonnost QuickSortu má však vliv i složitost porovnání dvou řetězců, která v případě řetězců začínajících na stejný dlouhý prefix vede až na $O(n)$ pro každé z $O(n^2)$ porovnání.
 - V nejhorším případě (řetězec stejných symbolů) dostáváme extrémně neefektivní algoritmus s $O(n^3)$.
- Řešení:
 - Burrows a Wheeler si všimnuli, že tento problém lze eliminovat zavedením zarážky (\$) a využitím struktury *Suffix tree* (*Suffix array*). Poté lze získat algoritmus s paměťovou i časovou náročností odpovídající $O(n)$. Konstantní faktor je však pro praktické hodnoty n příliš velký. Burrows a Wheeler proto navrhuji použít modifikovanou verzi QuickSort, která aplikuje RadixSort na první dva znaky každého klíče a bere ohled na speciální případ, kdy oba znaky jsou shodné.
 - Jinou možností je použít RLE kódování na vstupu, čímž eliminujeme problém s výskytem sekvencí stejných symbolů. RLE může mít i pozitivní efekt na kompresní účinnost. Ztrácíme však možnost vyhledávání řetězců v komprimovaných datech, což je např. podstatná funkcionalita pro kompresi genomu.



Vztah BWT a Suffix Array

- *Suffix array (SA)* je datová struktura odpovídající lexikograficky seřazenému poli všech sufixů určitého řetězce, která umožňuje efektivní realizaci fulltextového vyhledávání.
 - Pokud řetězec S končí symbolem, který je lexikograficky menší než ostatní symboly, pak výsledek BWT odpovídá Suffix array, konkrétně $L[i] = S[SA[i] - 1]$
 - Existují efektivní algoritmy umožňující konstrukci SA s čas. složitostí $O(n)$ nebo $O(n \log n)$

```
$aardvark
aardvark$
ardvark$a
ark$aardv
dvark$aar
k$aardvar
rdvark$aaa
rk$aardvaa
vark$aardd
```

BWT Matice M

$S = \text{aardvark\$}$
123456789

```
$
aardvark$
ardvark$
ark$
dvark$
k$
rdvark$
rk$
vark$
```

sufixy získané odstraněním
znaků za \$

sufixy zůstávají
seřazený neboť
\$ předchází
ostatní symboly

```
9
1
2
6
4
8
3
7
5
```

Suffix array (SA)
odpovídající počáteční
pozici sufixu v S

```
S[9-1]=k
S[1-1]=$   $ je
S[2-1]=a   speciální
S[6-1]=v   případ
S[4-1]=r
S[8-1]=r
S[3-1]=a
S[7-1]=a
S[5-1]=d
```

$SA[i] - 1$
odpovídá pozici
znaku $L[i]$ v BWT



Burrows-Wheeler transformace (implementace)

■ Zpětná BWT transformace

- LF mapping založený na stabilní řadící metodě

```
import operator

def iBWT(I, L):
    n = len(L)

    # compute the first row of the permutations
    # and keep the mapping from L to F
    F = sorted([(i, l) for i, l in enumerate(L)],
               key=operator.itemgetter(1))
    # (sort stably by second item)

    # compute the mapping from F to L
    T = [None for i in range(n)]
    for k, (j, _) in enumerate(F):
        T[j] = k

    # get the original text
    S = [None for i in range(n)]
    for i in range(0, n):
        S[n-i-1] = L[I]
        I = T[I]

    return ''.join(S)
```

Příklad pro vstup $(I, L) = (8, \text{'TSSHHIIITS'})$

$n = 10$

$F = [(3, \text{'H'}), (4, \text{'H'}), (5, \text{'I'}), (6, \text{'I'}), (7, \text{'I'}),$
 $(1, \text{'S'}), (2, \text{'S'}), (9, \text{'S'}), (0, \text{'T'}), (8, \text{'T'})]$

$T = [8, 5, 6, 0, 1, 2, 3, 4, 9, 7]$

$I = [8, 9, 7, 4, 1, 5, 2, 6, 3, 0]$

$S = [\text{'T'}, \text{'H'}, \text{'I'}, \text{'S'}, \text{'I'}, \text{'S'}, \text{'H'}, \text{'I'}, \text{'S'}, \text{'T'}]$

i^{-1}	k	F	L	i	$T = k^{-1}$
3	0	H ISISHIST T	T	0	8
4	1	H ISTTHISI S	S	1	5
5	2	I SHISTTHI S	S	2	6
6	3	I SISHISTT H	H	3	0
7	4	I STTHISIS H	H	4	1
1	5	S HISTTHIS I	I	5	2
2	6	S ISHISTTH I	I	6	3
9	7	S TTHISISH I	I	7	4
0	8	T HISISHIS T	T	8	9 ←
8	9	T THISISHI S	S	9	7



Burrows-Wheeler transformace (implementace)

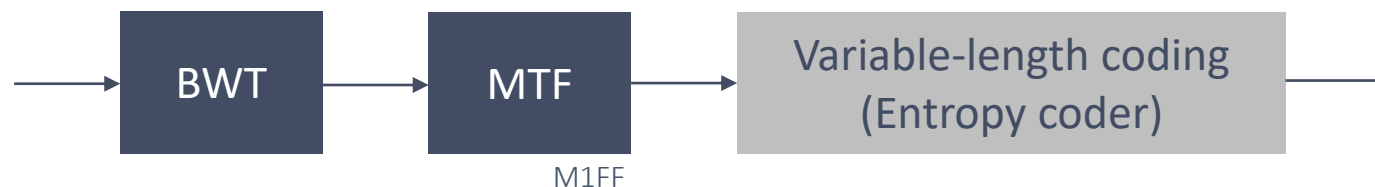
- Zpětnou BW transformaci lze formálně popsat následovně.
- Mějme na vstupu řetězec L délky n a index I . Původní řetězec S lze získat takto:
 1. První sloupec uspořádané matice (sloupec F) se zkonstruuje z L .
Je to jednoduchý postup, protože F i L obsahují stejné symboly (oba jsou permutace S). Dekodér pouze abecedně seřadí řetězec L stabilním řadicím algoritmem a dostane F .
 2. Během seřazování L dekodér připraví pomocné pole T , které ukazuje vztahy mezi prvky L a F .
Platí, že $T[i]$ odpovídá pozici symbolu $L[i]$ v F , přičemž $i = 0, 1, \dots, n - 1$.
 3. Řetězec F už nepotřebujeme. Dekodér použije k rekonstrukci S údaje z L, I, T podle vztahů:

$$S[n - 1 - i] = L[T^i[I]], \text{ pro } i = 0, 1, \dots, n - 1, \text{ kde}$$

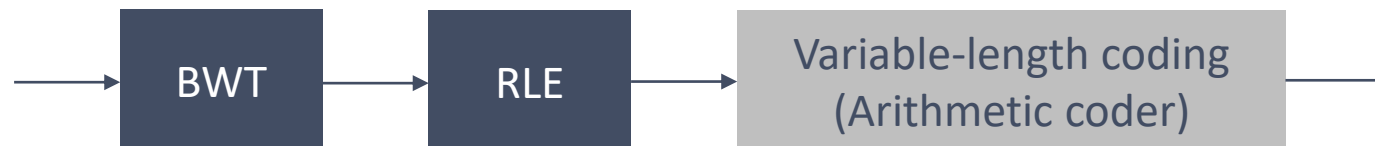
$$T^0[j] = j \text{ a } T^{i+1}[j] = T[T^i[j]]$$

Burrows-Wheeler transformace – použití v oblasti komprese dat

- V literatuře najdeme řadu technik snažících se využít vlastnosti sekvencí získaných BWT
 - V původní implementaci Burrows a Wheeler používali MTF transformaci, která těží z výskytu opakujících se symbolů.



- Následně se začaly objevovat kombinace BWT a RLE, např. RLEAc (Ferragina, 2006) používá kombinaci RLE s aritmetickým kóděm, aby dosáhl co nejvyšší úrovně komprese při zachování rozumné rychlosti zpracování dat. Kombinace s RLE jsou schopny konkurovat pokročilejším a výpočetně náročnějším technikám.



■ Využití BWT

- BWT lze použít k zvýšení kompresní účinnosti entropické komprese (compression boosting), kdy kompresor nultého řádu může být zavedením BWT povýšen na kompresor vyššího řádu.
- BWT lze však využít také pro vyhledávání (compressed text index)

Burrows-Wheeler transformace – kompresní účinnost

- Kompresní účinnost různých technik využívajících BWT vyjádřená v průměrném počtu bitů připadajících na jeden znak vyhodnocená na korpusu Calgary
 - MTF v kombinaci s Huffmanovým kódováním (Moffat, 1987) dosahuje 3.24 bpc (viz úvodní slidy)

Author	Date	Algorithm	bpc
Burrows and Wheeler	1994	16 000 Huffman blocks	2.43
Fenwick	1995	Order-0 arithmetic	2.52
Fenwick	1996	Structured arithmetic	2.34
Balkenhol <i>et al.</i>	1998	Cascaded arithmetic	2.30
Fenwick	1998	Sticky MTF, structured arithmetic	2.30
Balkenhol and Shtarkov*	1999	Cascaded arithmetic	2.26
Deorowicz	2000	Arithmetic contexts	2.27
Deorowicz*	2002	Weighted frequency count	2.25
Seward (BZIP2)	2000	Huffman blocks	2.37
Wirth	2001	No MTF; PPM contexts	2.35
Arnavut*	2002	Inversion ranks	2.30
Fenwick	2002	VL codes; 1000 blocks	2.57

* state of the art in mid 2002

- Vylepšit účinnost původní verze BWT je poměrně obtížné. Rozdíl oproti verzi z roku 1994 je cca 7%. Zatímco Burrows a Wheeler a později Seward v BZIP2 používá Huffmanovo kódování, ostatní autoři používají mnohem komplikovanější a časově náročnější přístupy.

Využití BWT v BZIP2

- BWT je použita v kompresním nástroji bzip2 (Seward, 1996-2000), který je určen ke kompresi souborů a je rozšířen zejména v prostředí Linux.
- Bzip2 dosahuje lepších kompresních poměrů oproti gzip (zip, gz), ale je mnohem pomalejší.

Program	Version	Size	Compression ratio	Time (s)			Peak MEM usage (MB)	
				Compression	Decompression	Total	Compression	Decompression
7-zip	9.12b	805 116 998	24.4 %	891	103	16m 36s	199	21
bzip2	1.0.5	1 125 342 869	34.1 %	468	167	10m 35s	9	5
gzip	1.3.2003	1 240 841 343	37.6 %	141	34	2m 56s	2	2

<https://web.archive.org/web/20160424151609/http://compressionratings.com/comp.cgi?7-zip+9.12b++bzip2+1.0.5++gzip+1.3.3+-5>

- Použitý přístup:
 - Metoda zpracovává vstupní soubor po blocích o velikosti mezi 100kB a 900 kB (best mode).
 - Vlastní komprese je dosažena pomocí RLE.
 - Na každý blok je aplikováno 5 fází: nejprve je vstup předzpracován pomocí obecného RLE, poté se aplikuje BWT následovaná MTF, další RLE (RLE-0) a výsledek je zakódován pomocí Huffmanova kódu.



BZIP2

- RLE v první fázi používá metodu bez explicitní značky. Sekvence 4 až 255 symbolů je nahrazena prvními čtyřmi symboly následovanými bytem definujícím počet opakování zmenšený o 4. (V nejhorším případě dojde k expanzi 1.25x, v nejlepším k redukci 51.8x)
 - Např. řetězec "AAAAAABBBBCCCD" je zakódován jako sekvence "AAAA\x03BBBB\x00CCCD".
 - Řetězec "BBAAAA" je zakódován jako "BBAAAA\x00".
- V druhé fázi se používá BWT využívající strukturu *suffix array*.
- V třetí fázi je aplikována MTF, která typicky produkuje sekvence nulových hodnot.
- Ve čtvrté fázi je použita metoda RLE-0, která má za cíl kompaktněji reprezentovat sekvence opakujících se nulových hodnot. Používá se speciální symbol RUNA a nula je nahrazena symbolem RUNB.
- V poslední fázi se provede zakódování pomocí Huffmanova kódu, který je sestaven na základě znalosti četnosti výskytů hodnot získaných MTF. Počet uzlů Huffmanova stromu odpovídá počtu různých hodnot MTF (bez hodnoty nula, která je nahrazena značkou RUNB) plus značka RUNA a EOB (End-of-block).

Očekávané znalosti

■ RLE

- Rozumět principu RLE, umět uvést kostru algoritmu, umět odvodit kompresní účinnost.
- Znat za jakých podmínek RLE pracuje výhodně.
- Rozumět principu komprese černobílých obrazů a obrazů ve stupních šedi.
- Umět uvést kostru algoritmu pro kompresi černobílých obrazů.

■ MTF

- Znat princip MTF, umět uvést kostru algoritmu
- Znat způsob využití MTF pro účely komprese, výhody a nevýhody MTF

■ BWT

- Znat princip dopředné i zpětné BWT, umět zrekonstruovat původní řetězec
- Znat způsob využití BWT pro účely komprese, podmínky, kdy metoda pracuje dobře
- Orientovat se v problematice čas. složitosti, proč je nutné zavádět RLE

■ Kódy

- Umět zkonstruovat gama kód